technische universität
dortmund

Intermediate Report

Projektgruppe H-ReACT

H-ReACT
Humanoid Reactive Action & Control Training

Ahmet Yasin Kurt
Hao-Yi Andy Chen
Ivanna Romanovych
Maik Steiger
Malte Kaufhold
Marius Maximilian Hille
Minh-Nhat Vu
Niklas Wasserberg
Phillip Bronzel
Sarah Prahl
Simon Thomas Buschmann
Steffen Cintosun
Yakup Yazir

October 16, 2026

Supervisors:
Prof. Dr. Falk Howar
Dr. Oliver Urban
Jan Finke
Dominik Brämer
Nicolas Bach

AQUA automated quality assurance
Technische Universität Dortmund
Fakultät für Informatik
Lehrstuhl XIV für Software Engineering
Arbeitsgruppe AQUA
https://aqua.cs.tu-dortmund.de

Abstract

Humanoid Robotics are on the rise and recent events like the World Humanoid Robot Games in Beijing have caught the eye of the public. Yet, advancements in this field predominantly take place in Asia, while European countries still have to catch up.

We aim to design a reusable modular pipeline that facilitates the teaching process for Humanoid Robots. The pipeline includes initial task specific recording of motion capture data, which is then retargeted to joint motions of the humanoid model. This retageted data is leveraged in an imitation learning process, followed by a reinforcement learning phase that refines the learned policy. Finally, the resulting Neural Network is tested by deploying it on another physics engine than it was trained on, before deploying it on the real robot.

To validate our pipeline, we teach the Unitree G1 to catch a ball, which introduces an additional challenge of incorporating an object besides the robot itself. To reduce latency, we also use event cameras to detect the ball, as they reduce data overhead by only transmitting information about objects in motion.

Contents

Abstract i

1 Introduction 1
- 1.1 Motivation and Background . . . 1
- 1.2 Goals of the project group . . . 1
- 1.3 Thesis Structure . . . 1

2 The Unitree G1 3
- 2.1 Description of the Unitree G1 . . . 3
  - 2.1.1 General descripton . . . 3
  - 2.1.2 Technical characteristics . . . 4
  - 2.1.3 Remote control . . . 4
- 2.2 Inspire Dexterous Hands . . . 4

3 Pipeline for Training a Humanoid Robot 5
- 3.1 Overview . . . 5
  - 3.1.1 Problem Statement . . . 5
  - 3.1.2 Pipeline Architecture . . . 6
  - 3.1.3 Mapping of Frameworks to Pipeline Stages . . . 7
- 3.2 Reasoning . . . 7
- 3.3 Related Work . . . 8
  - 3.3.1 Learning-based Humanoid Whole-Body Control and Motion Tracking . . . 8
  - 3.3.2 Dynamic Interaction Tasks with Humanoids . . . 8
  - 3.3.3 Object Catching and Event-based Ball Perception . . . 9
  - 3.3.4 Simulation and Training Frameworks . . . 9

4 Motion Capturing with Vicon Shogun 11
- 4.1 Setup . . . 11
  - 4.1.1 Optical Motion-Capture System . . . 11
  - 4.1.2 Finger tracking with the MANUS gloves . . . 11
  - 4.1.3 Software Pipeline . . . 11
  - 4.1.4 Calibration and Capture preparation . . . 12
- 4.2 Recording Process . . . 12
  - 4.2.1 Recording procedure . . . 13
  - 4.2.2 Synchronization of Body and Hand Data . . . 13
  - 4.2.3 Trial Design . . . 13
  - 4.2.4 Real-Time Ball Tracking . . . 13

4.3 Postprocessing . . . 15
4.3.1 Export of the Hand Data . . . 15
4.3.2 Automated Merge in Shogun Post . . . 15
4.3.3 Error analysis while merging . . . 16
4.3.4 Final Batch Script . . . 16
4.4 Recording Results . . . 17
4.4.1 Resulting Artifacts . . . 18
4.4.2 Scope of the DataSet . . . 18
4.4.3 Data Quality . . . 18
4.4.4 Limitations . . . 19

5 Capturing the Ball Trajectory with Event Cameras 21
5.1 Introduction to Event Cameras and Reasons for Usage . . . 21
5.2 Setup and Tracking Pipeline . . . 21
5.3 Calibration . . . 22
5.4 Noise Filtering . . . 22
5.5 Results . . . 23

6 Retargeting Motion Capture Data onto the Unitree G1 25
6.1 General Motion Retargeting (GMR) . . . 25
6.1.1 Project Description . . . 25
6.1.2 From initial approach to final retarget workflow . . . 26
6.2 Challenges in Retargeting . . . 27
6.2.1 Morphological and Kinematic Differences . . . 27
6.2.2 Geometrical Artifacts . . . 27
6.2.3 Physical feasibility . . . 28
6.3 Smoothing and Filtering of Retargeting Results . . . 28
6.3.1 General Architecture . . . 29
6.3.2 Detecting jumps between frames . . . 29
6.3.3 Detecting violations of joint limits . . . 29
6.3.4 Detecting collisions . . . 29
6.3.5 npz support . . . 30
6.3.6 Inspecting the robot's collisions . . . 30
6.3.7 Usage during our work . . . 31

7 Imitation and Reinforcement Learning with LocoMuJoCo 33
7.1 Introduction to LocoMuJoCo . . . 33
7.2 Defining Environments in LocoMuJoCo . . . 33
7.3 Learning on self-recorded data . . . 36
7.4 Incorporating Ball Trajectories into the Imitation Learning Process . . . 36
7.5 Imitation Learning Results . . . 36
7.6 From Imitation Learning to Reinforcement Learning . . . 37
7.6.1 Kullback-Leibler Regularization . . . 38
7.6.2 Introducing diverse Ball Throws . . . 38

7.6.3 Fitting Rewards to the Catching Process . . . 39
7.6.4 Reinforcement Learning Results . . . 41

8 Reinforcement Learning with Gymnasium via MuJoCo 43
8.1 Introduction to Gymnasium . . . 43
8.2 Creating Gymnasium-Environment . . . 43
8.3 Designing Reward-Functions . . . 43
8.3.1 Two-Layer Reward Architecture . . . 44
8.3.2 Lower-Level Regularization ($\mathscr{R}_{\text{lower}}$) . . . 44
8.3.3 Upper-Level Task & Curriculum Learning ($\mathscr{R}_{\text{upper}}$) . . . 44
8.4 Challenges and Results . . . 45
8.4.1 Starting with PPO . . . 45
8.4.2 Switching to SAC . . . 45
8.4.3 Final Results . . . 45

9 Reinforcement Learning and Motion Imitation using mjlab for Unitree Robots 47
9.1 Introduction to mjlab . . . 47
9.2 Defining the G1 Environment . . . 48
9.3 Motion Imitation . . . 48
9.4 Reinforcement Learning for Catching . . . 49
9.4.1 Designing Reward Functions . . . 49
9.5 Challenges and Results . . . 50

10 Unitree RL Lab on top of Isaac Lab 53
10.1 Introduction to Isaac Lab . . . 53
10.1.1 Isaac Sim and Isaac Lab . . . 53
10.1.2 Manager-based Environments . . . 53
10.2 The unitree_rl_lab Framework . . . 54
10.3 Integration of the Ball Catching Task . . . 55
10.3.1 Motion Data Preparation . . . 56
10.4 Training Approaches . . . 56
10.5 Course of the Work and Challenges . . . 57
10.6 Outlook . . . 58

11 Deployment of Policies 59
11.1 Sim-to-Sim . . . 59
11.1.1 Isaac Lab to MuJoCo . . . 59
11.1.2 MuJoCo to Isaac Lab . . . 60
11.2 Sim-to-Real . . . 60
11.2.1 Isaac Lab to Real . . . 60
11.2.2 MuJoCo to Real . . . 60

12 Conclusion 61

Bibliography 66

1 Introduction

1.1 Motivation and Background

In this report, we present our approach to teaching the Unitree G1 humanoid robot to catch a thrown ball while standing. We use the Vicon motion capture system to track the ball and provide its position to the trained policy.

At the beginning of the project, the papers found in our literature search focused on catching with quadruped robots [6]. We had not found a comparable approach for ball catching with a standing humanoid robot. During the project, the workshop paper Catch It All! (wurde an einem workshop auf der RSS 2026 vorgestellt idk ob serioues genug) presented an approach for catching different objects with a Unitree G1 [23]. However, their robot remained seated and used one arm and hand for catching. Their approach did not include whole body control, which is an additional challenge in our project.

We use the Vicon motion capture system to track the ball. This allows us to get very accurate position measurements. We also try out event cameras because they can capture fast movements with high temporal resolution. They could later provide an alternative way to track the ball instead of using the external Vicon system.

The project is carried out in cooperation with Fraunhofer IML.

1.2 Goals of the project group

The overall goal of the project is to build a reusable pipeline. For us as students, the specific goal is to get the robot to catch at least 50% of the balls in real-world tests and 90% in simulation while remaining standing.

To achieve this, we aim to explore different training approaches, including reinforcement learning on its own and imitation learning followed by reinforcement learning. For imitation learning, the goal is to use recorded human catching movements and retarget them to the robot. Vicon provides the motion capture data and will also be used to track the ball and the robot's position during real-world tests.

Another goal is to test the trained policies in different simulation environments before deploying them on the real robot. So Sim-to-sim testing to identify problems and help prepare for sim-to-real deployment.

1.3 Thesis Structure

We start by introducing the Unitree G1 used in our project (ch. 2). We then describe our pipeline to give an overview of the workflow and explain which tools were used for each step (ch. 3). After that, we look at the individual parts in more detail, starting with how we used Vicon Shogun to record the movements and what needed to be considered during recording (ch. 4). This is followed by our experiments with event cameras as another approach to ball tracking (ch. 5). We finish this part with motion retargeting,

explaining how we used GMR and which adjustments were needed to map the recorded movements onto the robot (ch. 6).

We then move on to imitation learning and reinforcement learning. Here, we describe the different approaches we tried, including reinforcement learning with and without prior imitation learning. We also cover the different simulation environments and frameworks used, including LocoMuJoCo (ch. 7), Gymnasium with MuJoCo (ch. 8), mjlab (ch. 9), and Unitree RL Lab based on Isaac Lab (ch. 10).

Finally, we cover sim-to-sim testing and sim-to-real deployment (ch. 11) before concluding with the overall results and possible next steps (ch. 12).

2 The Unitree G1

The Unitree G1 is a humanoid robot by Unitree Robotics. Standing, it is about 1.32m tall and weighs about 35kg with its battery [28]. Each leg has six degrees of freedom, each arm has five, and the waist has at least one. The basic version therefore has 23 degrees of freedom. The G1-EDU version can have 23 to 43, depending on optional waist and wrist joints and dexterous hands [28]. This project uses the 29-DoF configuration, which has a three-axis waist and three-axis wrists. The joints are driven by motors developed by Unitree with dual encoders. The largest joint reaches a peak torque of up to 120Nm [28]. For perception, the head carries a Livox MID-360 LiDAR and an Intel RealSense D435i depth camera [28]. The robot has two on-board computers. A closed motion control unit runs Unitree's own controller. The EDU version adds a development unit (NVIDIA Jetson Orin NX) that is open for custom software [28]. With its 9000mAh battery, the robot runs for about two hours [29]. This chapter describes the parts of the G1 that matter for training and deploying policies.

2.1 Description of the Unitree G1

2.1.1 General descripton

G1 is an integrated humanoid intelligent entity featuring advanced motion control algorithms, Al learning capabilities, and a high cost-performance ratio. With exceptional athletic performance, it enables the execution of complex actions, catering to the research, education, and entertainment sectors. It pioneers the popularization and innovation of humanoid robot technology. Designed with an all-joint hollow internal wiring system, it features dual-encoder equipped joint encoders and a localized forced air-cooling system, significantly enhancing operational precision and runtime stability.

2.1.2 Technical characteristics

Height, Width and Thickness (Stand)	1320x450x200mm
Height Width and Thickness (Fold)	690x450x300mm
Weight (With Battery)	About 35 kg
Total Degrees of Freedom (Joint Freedom)	23
Single Leg Degrees of Freedom	6
Waist Degrees of Freedom	1
Single Arm Degrees of Freedom	5
Full Joint Hollow Electrical Routing	YES
Joint Encoder	Dual encoder
Cooling System	Local air cooling
Power Supply	13 String lithium battery
Basic Computing Power	8-Core high-performance CPU
Sensing Sensor	Depth Camera+3D LiDAR
Microphone Array	Noise Cancellation, Echo Cancellation
Speaker	Stereo, 5W Power
Wireless connectivity	WiFi 6, Bluetooth 5.2

2.1.3 Remote control

The R3 remote control is part of the G1 remote control module. The remote control handle is equipped with a digital transmission module and a Bluetooth module. The robot and the remote control are bound through the Unitrec Explore App, and once successfully bound, they can be connected upon powering on. It can control the robot to achieve 3-axis posture and 3-axis position stability when standing, and can also control the robot to move forward, backward, left, right, turn in place, and walk according to certain rules (straight line, circular, linear, rectangular) on flat ground, as well as go up and down slopes/steps.

2.2 Inspire Dexterous Hands

The Dexterous Hands (RH56DFTP) offer six degrees of freedom, including a rotating and bending thumb, alongside a built-in pressure sensor. They communicate using RS485, CAN bus, or Modbus RTU protocols by reading and writing to specific memory registers. While action sequences can be designed through a GUI, operating the hands via the API requires specific control logic. The API relies on three core commands to manage the hardware. “Act Angle” uses a scale from 0 to 1000 to set the finger’s target position, where 0 is fully open and 1000 is fully closed. “Act Force” uses the same 0 to 1000 scale to limit the maximum gripping strength. Finally, the “Read State” command monitors the current angle, power draw in mA, and temperature to prevent the hands from overheating.

3 Pipeline for Training a Humanoid Robot

The training process of a humanoid robot is complex with many subtasks, such as data collection, policy training, and deployment. This chapter serves as an overview over the entire training pipeline to visualize the training process in its entirety. Additionally, we explain, why we chose this specific workflow and introduce important related work our project is based on.

3.1 Overview

3.1.1 Problem Statement

We want to train a humanoid robot, the Unitree G1 (29 DoF), to catch a ball thrown by a human. The robot is not allowed to walk towards the ball but is instead required to remain standing and keep its balance. We control the robot with a single policy, which coordinates the whole body (whole-body control).

The policy receives the ball state. We aim to solve this task in simulation with ground-truth ball position data and in the real world with ball position tracked by Vicon (see ch. 4). As a lower latency alternative to motion capture, we also explore the use of event cameras (see ch. 5).

The minimum goal of this project is to achieve repeated catches in simulation with ground-truth data with a catching success rate of at least 90 % and to deploy the policy on the real robot with a success rate of at least 50 %. Beyond the task we aim to create a reusable and modular pipeline for training humanoid robots on new reactive tasks.

We are challenged by several factors: A short flight time of the ball leaves a very small time window to react. A high-dimensional action space and the interaction with a free object besides the robot itself make the task more difficult. Also, achieving a human-like catching motion is hard by reward design alone, but to use imitation learning, we need to record our own dataset of catching motions, as no existing dataset is available. Since there are morphological differences between the human and the robot, retargeting of these data is necessary. Finally, we need to overcome the sim-to-real gap, most notably the differences in physics, noise and latency of the ball measurement.

3.1.2 Pipeline Architecture

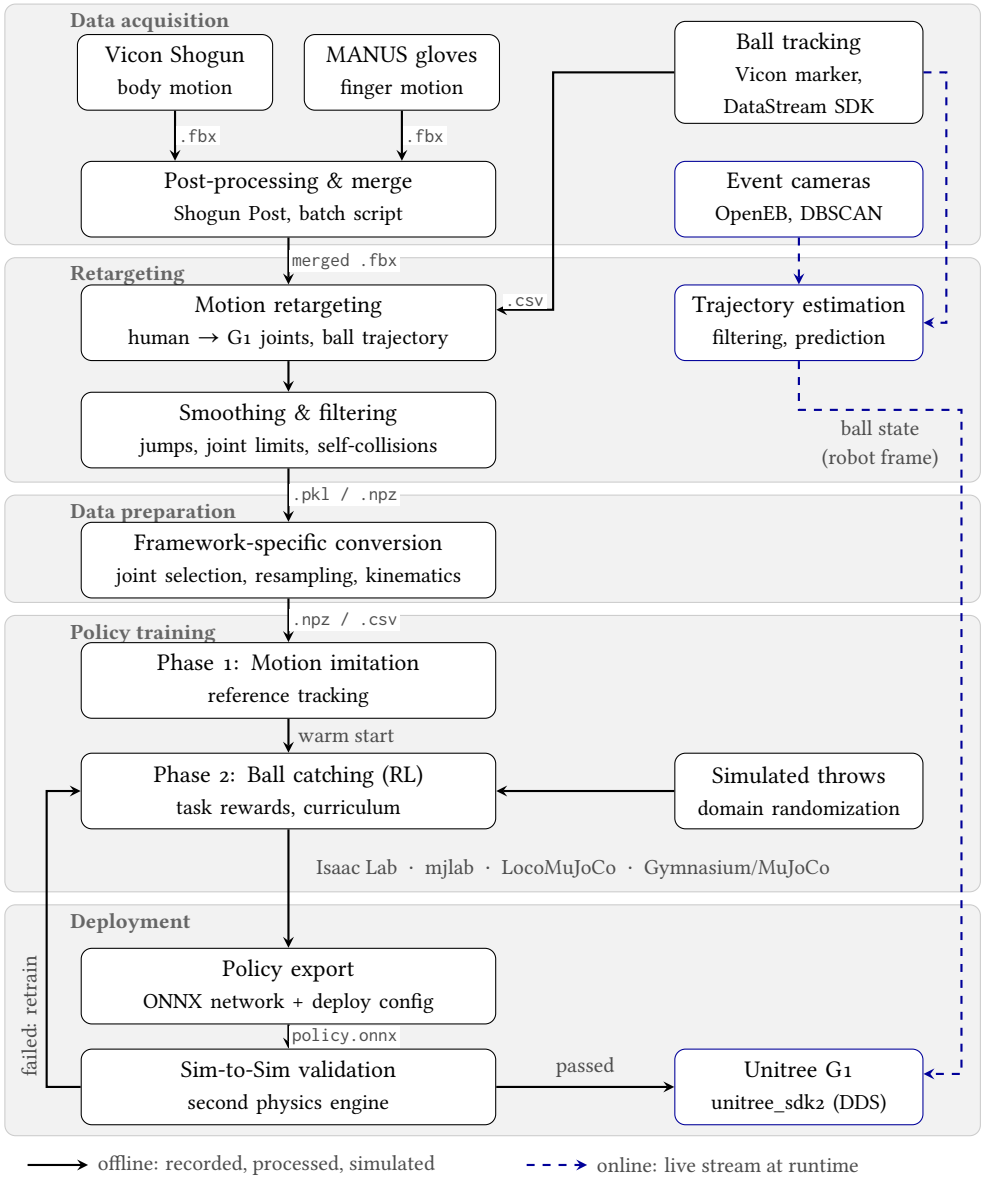


Figure 3.1: Overview of the H-ReACT training pipeline.

- Data acquisition: demonstrations with Vicon and MANUS gloves, ball as Vicon marker (ch. 4)
- Retargeting: motion and ball trajectory mapped onto the G1, then filtered (ch. 6)
- Data preparation: conversion into the format of each training framework (ch. 7–10)
- Policy training: motion imitation, then RL for catching; four frameworks in parallel (ch. 7–10)
- Deployment: ONNX export, Sim-to-Sim validation, then the real robot (ch. 11)
- Online (blue): live ball state from Vicon, later from event cameras (ch. 4, 5)

3.1.3 Mapping of Frameworks to Pipeline Stages

We split the development of the pipeline into subgroups, each responsible for a specific stage of the pipeline. We explore the training stage in four different frameworks in parallel, instead of committing to one early. The reason for this is that we have little prior experience in the group, and it is unclear which framework is best suited for the catching task. The following table serves as an overview of the pipeline stages, the frameworks and tools used, and the responsible group members.

Stage	Frameworks and tools	Members	Ch.
Motion capture	Vicon Shogun, MANUS gloves, ball tracking via Vicon DataStream SDK	Bronzel, Cintosun, Kurt, Romanovych, Yazir	4
Event cameras	OpenEB, DBSCAN clustering	Buschmann, Yazir	5
Retargeting	GMR, Blender, smoothing and filtering tool	Kaufhold, Vu, Wasserberg	6
Training	LocoMuJoCo: imitation + reinforcement learning	Chen, Prahl	7
	Gymnasium + MuJoCo: reinforcement learning only (PPO, SAC)	Buschmann, Prahl	8
	mjlab: motion imitation + reinforcement learning (RSL-RL)	Bronzel	9
	Isaac Lab (unitree_rl_lab): motion imitation + reinforcement learning (RSL-RL)	Hille, Steiger	10
Deployment	unitree_rl_lab deploy program, unitree_mujoco, unitree_sdk2	Steiger	11

Table 3.1: Pipeline stages with the frameworks and tools used and the responsible group members.

3.2 Reasoning

Learning complex human behavior from pure reinforcement learning has been shown to be difficult. Even with highly designed reward functions one usually cannot achieve a replication of how humans act. Since we aim to teach a humanoid robot the task of catching a ball, which is reliant on smooth arm and hand movements, we assume demonstrations of this task via imitation learning are valuable to successfully learning a policy.

Imitation learning needs large datasets that demonstrate the desired task. However, there are no pre-existing datasets that depict the task of catching a ball. Thus, we have to record our own dataset with the motion capture software Vicon Shogun. To also capture the fine finger motions we utilize Manus gloves. Since the recorded data tends to be noisy, additional filtering of this motion data is necessary.

The recorded motion data can not be directly leveraged by imitation learning, since the physical constitution of the humanoid robot and the human that demonstrated the task differ. For this reason the recorded motion data must first be transferred to robot motion data via retargeting. This translates the motion of human limbs directly to the joint angles of the humanoid robot, in our case the Unitree G1.

Now the retargeted motion can be leveraged by imitation learning algorithms. These algorithms oper-

ate on simulations of the robot instead of on the real robot for safety and economical reasons. To explore different options, we choose to use two physics engines separately: MuJoCo and Isaac Lab.

Since pure imitation learning often yields poor generalization, we further refine the policies with additional reinforcement learning. This step should create more robust policies and also make up for missing diversity in the imitation training data.

Before deploying a policy on the real robot, it is recommended to test the policy in another physics engine than the policy was trained on, to ensure the policies generalization ability. We first simulate policies trained in MuJoCo in Isaac Lab and vice versa, before testing them on the real robot.

3.3 Related Work

To give an overview of the work our approach builds on, we present related work in the following sections grouped by topic. We describe the technical details of the actually used ideas in the respective chapters.

3.3.1 Learning-based Humanoid Whole-Body Control and Motion Tracking

DeepMimic and BeyondMimic are two already existing approaches to learning-based humanoid whole-body control and motion tracking. DeepMimic formulates imitation learning as reinforcement learning with rewards for tracking a reference motion and can optionally combine this with a task objective [16]. This is the basis for the imitation approach in LocoMuJoCo (see ch. 7). BeyondMimic extends this idea to real humanoid robots with sim-to-real transfer and proposes further improvements for versatile control [9]. This is used as the imitation approach in mjlab and unitree_rl_lab (see ch. 9 and 10), although only the motion-tracking part.

Retargeting is a prerequisite for these approaches, since they require a motion to reference. Moreover, the quality of the retargeted reference motion strongly affects the tracking performance [3]. GMR can be used to map human motion onto humanoids, in our case the Unitree G1 [41]. An alternative to that is neural retargeting [43]. We describe our retargeting workflow in (see ch. 6). In combination this forms a complete imitation learning pipeline for humanoid robots, which we use as a basis for our approach.

3.3.2 Dynamic Interaction Tasks with Humanoids

LATENT is a recent work that trains the Unitree G1 to return tennis balls to a target location. It uses a similar approach to ours, with imitation learning from fragments of human tennis playing motion and reinforcement learning for task optimization [42]. The creators were able to achieve a real-world multi-shot rally with a human second player after their sim-to-real transfer. This setup is close to our approach, since it uses the same robot, a motion capture reference, and interaction with a flying ball. The main difference is the task being hitting instead of catching and the use of a racket instead of a dexterous hand or respective mount. To this current date, only the motion-tracking stage of this work is publicly released.

Catch It All! is a recent work that trains a Unitree G1 to catch objects with a dexterous hand [23]. There the robot stays seated and catches only one-handed. Consequently, the task can ignore the problem of whole-body balance, which is a major challenge in our task. The success-rate of the real-world deployment is with at max 38.9% (underhand catch) significantly lower than the one we aim to achieve.

To our knowledge, there is no publicly available approach for catching with a standing humanoid robot using whole-body control and a human motion reference.

3.3.3 Object Catching and Event-based Ball Perception

The event camera itself is a sensor that reports asynchronous per-pixel brightness changes instead of complete frames. This provides low latency, high temporal resolution, and a high dynamic range [8].

This provides low latency, high temporal resolution, and a high dynamic range [8].

Researchers have explored the task of catching objects with a quadrupedal robot using an event camera [7]. They achieved a catching success-rate of 83% in real-world experiments. Transferring this idea to a humanoid robot was a starting point for our project proposal.

Also, there are several works on event-based ball tracking in sports. For volleyball, a real-time ball tracking method has been proposed with an event camera and without machine learning [39]. For table tennis, an egocentric event-camera-based ball trajectory prediction method has been proposed [2]. At a later stage of our project this becomes relevant to us as reference for ball detection and trajectory prediction alternatively to Vicon.

3.3.4 Simulation and Training Frameworks

MuJoCo is a physics engine for contact-rich robotics [26]. It serves as a basis for most of our frameworks: LocoMuJoCo (MJX/JAX) [1], Gymnasium environments [27] and mjlab (MuJoCo Warp) [40]. The benefit of using MuJoCo is that it is widely used while being not exclusively dependent on NVIDIA GPUs. Therefore, it is easy to run on a variety of hardware, including our local workstations.

Isaac Lab is a GPU-accelerated robot learning framework on top of NVIDIA Isaac Sim, based on PhysX [14]. It provides similar functionality to MuJoCo, but is optimized for NVIDIA GPUs and allows for massively parallel environments. unitree_rl_lab is a framework built on top of Isaac Lab that provides tasks and deployment code for Unitree robots [30]. An implementation of our catching task into unitree_rl_lab is one of our tried approaches (see ch. 10).

PPO is the algorithm mainly used for reinforcement learning in the frameworks we use [21]. RSL-RL provides the implementation of PPO and other algorithms in mjlab and Isaac Lab [22]. Stable-Baselines3 provides reinforcement learning algorithms for Gymnasium [18].

Two independent physics engines (MuJoCo, PhysX) make sim-to-sim validation possible (see ch. 11). Therefore, we can test the generalization ability of our policies by sim-to-sim transfer before deploying them onto the real G1 robot.

4 Motion Capturing with Vicon Shogun

The dataset used in this project was created by using full-body tracking from a human demonstrator, combined with high-resolution finger data and a trajectory of the thrown ball. The full-body motion is captured by using the marker-based Vicon Shogun system and recorded via *Shogun Live*. The finger movements are tracked using the non-haptic MANUS Metagloves via the *MANUS Core* software. Both recordings are coordinated using capture triggers and then merged into a single skeleton hierarchy during post-processing in *Shogun Post*. The trajectory of the thrown ball gets determined using the *Vicon DataStream SDK* by tracking the ball as an unlabeled marker and recording its trajectory separately in a CSV file or by publishing it through *MQTT*. This is done through a separate Python script.

The objective of this process is to create a consistent and reproducible dataset of body, finger, and ball motion that can be used as a reference point for retargeting onto the humanoid robot. This functions as a basis for subsequent learning, such as *Imitation Learning*. This chapter describes the systems and processes used in order to achieve this named goal.

4.1 Setup

4.1.1 Optical Motion-Capture System

For the data collection, a marker-based infrared optical motion-capture system from Vicon is used. Its measurement principle is based on reconstructing retroreflective marker positions from multiple camera views [36]. Here, each camera is equipped with a ring of infrared LEDs, whose light is reflected by the marker spheres attached to the subject back into the camera optics. A band-pass filter, which is placed in front of the sensor, helps the markers appear as high-contrast blobs in the camera image [35, pp. 31, 42].

The centers of the markers are then determined internally by the cameras with sub-pixel resolution [35, pp. 23, 164]. Subsequently, the three-dimensional marker positions are reconstructed from the two-dimensional observations of multiple cameras by intersecting the corresponding viewing rays. This requires the marker to be visible from multiple camera views [36].

All recordings are captured at a system frame rate of 120 fps.

4.1.2 Finger tracking with the MANUS gloves

While the Vicon system is used for the full-body motion tracking of the performer, optical finger markers can become occluded during the catch. For this reason, the setup introduces the *MANUS gloves* to track the individual finger movements. The glove data is processed and recorded using *MANUS Core* [10, 11].

4.1.3 Software Pipeline

Several software components are used for acquisition and processing of the different data streams.

Shogun Live is used for the calibration, recording, real-time reconstruction, labeling, and solving, as well as data streaming via the *Vicon DataStream SDK*. Each recording is stored as a `.mcp` file containing the recorded motion and the calibrated subject [36].

MANUS Core records the corresponding hand and finger data and saves them as a `.mrec` file, which gets exported to `.fbx` for the later integration into the body recordings [11].

Shogun Post is used for offline post-processing. Its main functionality here is to merge the body and hand data and export the resulting motion [13]. The merge is automated using the integrated *Hierarchical Scripting Language* (HSL) [34], as described in Section 4.3.

For real-time access, Vicon provides the *DataStream SDK*. This enables a separate Python script to access even unlabeled marker positions to track the ball during the throw [37].

4.1.4 Calibration and Capture preparation

The quality of the calibration directly affects the measurement accuracy of the system. Thus, calibration is a prerequisite for using the recorded data as reference data. This procedure can be split into four phases:

1. **Camera setup** The recordings were conducted at the ZFT-Hall (Fraunhofer IML, Dortmund). In sum, 52 cameras are available. However, since the catch movement is done on a fixed position, only a few of those are actually in use. Four additional cameras are positioned near this area at a lower viewing angle to improve visibility and reduce possible occlusions especially during the catch-motion.

2. **System calibration** A T-shaped calibration wand (*Active Wand*), which contains active LEDs with known geometry, is moved throughout the complete capture volume. These observations are used to calibrate the camera system. The world-coordinate origin and axis orientation is then established by placing the wand at the predefined location on the floor [36, pp. 77–88].

3. **Subject calibration** The human performer is equipped with a marker set according to the corresponding labeling template (ref. Bild Anzug oder so). First, he assumes an A-pose to initialize the subject in *Shogun Live*. The performer is then supposed to carry out a *range of motion* (ROM) sequence. During this, the relevant joints and body parts/segments are moved through their respective ROM. *Shogun Live* uses the observed marker positions to adapt the subject model to the individual body geometry of the performer [36, p. 146].

4. **MANUS Gloves Calibration** The MANUS gloves are calibrated for the user using *MANUS Core* by executing the calibration poses required by the software. This calibration is used to adapt the hand model to the user before recording [10].

4.2 Recording Process

Each session follows a fixed order of steps in order to keep the individual tasks comparable to each other. The camera setup, the system and subject calibration, as well as MANUS gloves calibration described in section 4.1.4 are carried out first.

The resulting calibrated subject, named `Pro3` remains valid for the entire session as long as no markers are displaced and the performer does not change.

4.2.1 Recording procedure

The recordings follow a one-take-per-throw scheme. Each recording contains exactly one throw of the ball together with the corresponding catching motion of the performer. The recording starts shortly before the ball is thrown and ends after the performer catches it.

However, this scheme causes the following issue: The number of files becomes the limiting factor, since each take generates separate body and hand files that require subsequent merging for use. To avoid repeated manual processing, this step was automated using the procedure described in Section 4.3.

Aborted or discarded takes are not renumbered. Consequently, gaps in the trial numbering are retained and handled by the post-processing script.

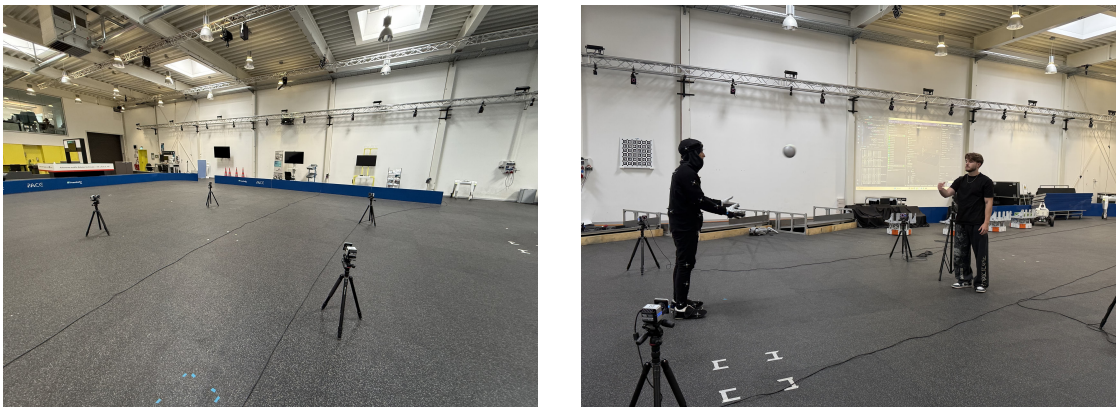
4.2.2 Synchronization of Body and Hand Data

All three parts, the *Shogun Live* full-body capture, the *MANUS Core* hand capture, and the ball trajectory script, are coordinated using Shogun's UDP-based capture triggering system [12]. This coordinates recording start and stop and associates the body, hand, and ball recordings through the capture name.

Merging and post-processing of these files is described in Section 4.3.

4.2.3 Trial Design

TODO



(a) Camera Setup

(b) Catch

Figure 4.1: todo

werfer und fänger stehen ca. 4 meter entfernt voneiannder gefangen wird von unten geworfen wird von unten?

wir nutzen die bereits bestehenden kameras die allerdings alle weit oben angebracht sind und daher noch 4 weitere die aus einem besseren winkel die hände und den ball auch von unten aufnehmen können.

der fänger steht leicht vor 2 von den 4 neuen kameras, der werfer zwischen 2 von den neuen bla bla

4.2.4 Real-Time Ball Tracking

The ball is covered in retroreflective tape and tracked as a single unlabeled marker. This provides a three-dimensional position measurement, but a single tracked point does not determine the ball's orientation. For our use case we only need the position and an estimate of its velocity. A separate Python script retrieves reconstructed unlabeled marker positions through *Vicon DataStream SDK* [37]. The resulting ball observations are published via MQTT for online use or saved to CSV files for training.

Because our ball is just one unlabeled marker, Vicon cannot differentiate it from other elements or reflections it finds, and thus the script needs to handle identifying the ball in the first place and then maintaining its identity across frames.

This is implemented through a state machine with two states: `SEARCHING` and `TRACKING`.



Figure 4.2: Ball with retroreflective tape applied.

Searching In the `SEARCHING` state, a fixed rigid body is registered and serves as a spatial reference object. Its root segment is queried per frame. The script computes the unlabeled marker closest to this reference. The candidate is accepted if its distance to the anchor falls below `acq_dist_mm`. The ball is brought into this acquisition zone before each throw, excluding candidates outside the zone. If the reference object is occluded or no unlabeled marker is present at all, the state remains unchanged and no candidate is accepted. This is visible through the terminal logs.

Tracking Once the ball is acquired, the reference point is no longer used. In the state `TRACKING`, the marker closest to the last known ball position is selected and accepted only if its distance stays below `track_dist_mm`. This threshold acts as a gate on the plausible displacement between two consecutive frames: At 120 fps, the threshold of 500 mm corresponds to approximately 60 m/s between consecutive frames, allowing fast ball motion while still rejecting more distant candidates.

However, a frame without an accepted candidate is not treated as a loss immediately. The tracker tolerates up to eight consecutive frames without an accepted detection before declaring tracking lost. At 120 Hz, this corresponds to approximately 67 ms. During this interval no new measured ball position is available. If this limit is exceeded, the script returns to `SEARCHING`, and the ball must be reacquired using the reference object.

Ballistic fit The tracked positions are then fitted over a sliding window of the last known positions using least squares:

$$x(t) = a_x t + b_x, \qquad y(t) = a_y t + b_y, \qquad z(t) = a_z t^2 + b_z t + c_z. \tag{4.1}$$

The fitted trajectory can be used to estimate future ball positions during flight. Here, t denotes time within the fitting window. The current training pipelines use the real positions rather than the fitted trajectory. Extrapolation after contact with the hands is unreliable because the contact changes the ball's dynamics.

Recording and output Recording is triggered automatically by the `CaptureStart` and `CaptureStop` UDP messages from *Shogun Live*. The capture name associates the ball data with the corresponding body and hand recordings. Each session produces `samples.csv` with tracked positions, velocities, and timestamps; `parabolas.csv` with accepted fit parameters; and `metadata.json` with the recording configuration.

Limitations The ball data uses workstation timestamps rather than Vicon timecodes, so recordings are associated by take but are not synchronized at the individual frame level.

4.3 Postprocessing

The objective of the post-processing stage is to produce a single consistent file that contains both the full body and finger motion of both hands within one common skeleton hierarchy. Based on the one-take-per-throw scheme described in Section 4.2.1, manual processing is inefficient, and thus, the complete post-processing procedure is carried out by scripts. This section explains the HSL script provided under Listing 4.1.

4.3.1 Export of the Hand Data

At first, all `.mrec` files are exported from *MANUS Core* to `.fbx` using the integrated batch exporter. The batch exporter supports multi-file exports, which allows all takes of a session to be processed in a single run [11]. The exported `.fbx` files follow the following naming scheme `<number>_<timestamp>_MANUS_MANUS_cm_user.fbx`. Afterwards, these need to be normalized to `<number>.fbx`, so that the trial number alone identifies the corresponding file.

4.3.2 Automated Merge in Shogun Post

The merge itself happens in *Shogun Post* using an HSL script. Instead of selecting the recordings manually, the script iterates over the range of trial numbers and checks whether the corresponding body recordings exist using `pathExists`. Missing body recordings are skipped automatically, allowing the batch-processing pipeline to handle gaps in the recording sequence without manual intervention.

The MANUS integration provides scripts for merging hand and body recordings in Shogun Post [13]. Our batch script uses the path-based `ImportManusFBXFile` call shown below, avoiding manual file selection for each take.

For importing the hand data to *Shogun Post*, our script uses the command `ImportManusFBXFile`, which accepts a direct path together with an identifier for the hand side (”`L`” or ”`R`”) and is called twice per take on the same FBX file. This command also removes the Vicon finger bones, their constraints and replaces the solving setup in order to remove the finger VRA. After this, it attaches the MANUS hand hierarchy to the existing `LeftHand` and `RightHand` bones.

However, HSL does not support any type of `catch` construct, which results in the following issue: If any error occurs during the run inside the loop, the entire batch run gets aborted. Therefore, instead of using an error handler, a precondition check is implemented. The labeling rate of the subject is queried via `getPercentSubjectLabeled`, and takes falling below a defined threshold are skipped. The affected trial numbers are collected and printed at the end of the run so that they can be inspected manually.

4.3.3 Error analysis while merging

An earlier version of this HSL script deleted the existing skeleton and imported a fresh skeleton template `FrontWaist.vss` and called `calibrateSolvingSubject`. This idea was based on the assumption that integrating the finger data requires a newly initialized skeleton. However, this version of the script created a reproducible deformation of the pelvis that varied in severity between recordings. The femur heads were placed too far medially, so that the thighs had to splay outward in order to reach the actual knee markers.

The analysis of this issue showed that the deformation was not caused by the import of the data itself but by the calibration step. Since `calibrateSolvingSubject` was executed on a single static frame of the respective take, no functional hip rotation was available to the solver. In this case, the hip joint centers cannot be determined from the observed motion and instead are estimated by regression from the pelvis markers of the template. This estimation does not reflect the actual individual anatomy of the performer, and depending on the pose present in the selected frame, it can diverge in either direction. This explains both the nature of the deformation and the fact that its severity differed from take to take.

The correction consisted exclusively in the removal of this part in the code. Since every `.mcp` file already contains a sufficient skeleton, with a subject that was properly calibrated during the ROM sequence, a recalibration is useless. The decisive observation is that the bone geometry, including the joint centers, is attached to the solving bones and not to the solving setup. Thus, the replacement of the solving setup performed by the MANUS import therefore leaves the calibrated proportions untouched.

4.3.4 Final Batch Script

Listing 4.1 shows the final version of the script. For every take, the results are saved as `.mcp`, which is the native working format for later editing in *Shogun Post*, and as `.fbx`, which acts as the exchange format for the subsequent retargeting pipeline.

```
string $mcpDir = "D:/H-ReACT 2/Aufnahmen/06_07_26/live";
string $fbxDir = "D:/H-ReACT 2/Aufnahmen/06_07_26/manus";
string $outputDir = "D:/H-ReACT 2/Aufnahmen/06_07_26/export";

int $startNum = 1;
int $endNum = 300; // upper bound; missing numbers are skipped
float $minLabeledPct = 99.0; // quality gate; 0.0 disables it

int $processed = 0;
int $merged = 0;
string $skipped = "";
int $num;

for ( $num = $startNum; $num <= $endNum; $num = $num+1 )
{
    string $mcp = $mcpDir + "/" + string( $num ) + ".mcp";

    if ( `pathExists $mcp` )
    {
        $processed = $processed + 1;
        print ( "Processing #" + string( $num ) + " -> " + $mcp );

        dataIssuesMap -enabled false;
        dataIssuesMap -enabled true;

```

```
        newFile;
        loadFile $mcp;

        SelectNextShelfTab;
        animRange 0 -start;

        select ;
        select "Camera1" -a;
        delete;

        // No deletion of the solving node, no template reload, no
        // recalibration: the calibration originates from the MCP file.
        setCurrentSubject Pro3;

        float $pct = `getPercentSubjectLabeled Pro3`;
        print ( " labeling Pro3: " + string( $pct ) + " %" );

        if ( $minLabeledPct > 0.0 && $pct < $minLabeledPct )
        {
            print ( " >>> SKIPPED #" + string( $num )
                   + " - insufficient labeling" );
            $skipped = $skipped + string( $num ) + " ";
        }
        else
        {
            // Left and right hand from the same FBX file
            string $fbx = $fbxDir + "/" + string( $num ) + ".fbx";
            ImportManusFBXFile $fbx "L";
            ImportManusFBXFile $fbx "R";

            string $outMcp =
                $outputDir + "/" + string( $num ) + "_merged.mcp";
            saveFile $outMcp;

            string $outFbx =
                $outputDir + "/" + string( $num ) + "_merged.fbx";
            saveFile $outFbx;

            $merged = $merged + 1;
        }
    }
}

print ( "Done. " + string( $processed ) + " files found, "
        + string( $merged ) + " merged." );

if ( $skipped != "" )
{
    print ( "SKIPPED (please check manually): " + $skipped );
}
```

Listing 4.1: `batch_merge_auto.hsl` – automated merge of body and hand data while preserving the calibration

4.4 Recording Results

This Section describes the dataset that results from the recording and post-processing procedure, as well as its quality and its limitations.

The result of the post-processing stage is one file per take that contains the full-body motion of the calibrated subject, together with the attached finger motion of both hands, which are recorded via *Manus Hands*, in a single hierarchy. Together with the CSV log of the ball trajectory and the trial metadata, this serves as the input dataset for the retargeting described in Chapter 6

4.4.1 Resulting Artifacts

Each take produces several files that originate from the three independent sources introduced above. *Shogun Live* records the full-body motion as a `.mcp` file, *MANUS Core* stores the corresponding hand and finger motion as a `.mrec` file, and a Python script writes the ball trajectory to a `.csv` file. Since the recordings in *Shogun Live* and *MANUS Core* are coupled through the triggered recording mode, the take identifier functions as the key to associate the body, hand, and ball recordings.

The result of the post-processing stage is one folder per take, containing the merged `.fbx` file of the full-body and finger data, as well as the `.csv` data of the trajectory from the ball. Table 4.1 summarizes the resulting files.

4.4.2 Scope of the DataSet

The dataset contains of … sessions with xy recordings per session. TODO. Also eig haben wir ja am ende nur ein vernünftiges dataset?

4.4.3 Data Quality

The quality of the individual takes differs. The labeling rate of the subject is therefore used as a quantitative criterion in the batch script. A take is only merged if at least 99% of the subjects's markers are correctly labeled at the queried frame [34]. This does not verify that the labels are correct or that coverage remains above the threshold throughout the take. The check is therefore an initial filter rather than a guarantee of data quality.

Two causes for a reduced labeling rate were observed. The first is occlusion of the markers on the back of the hand during the catching motion. The incidence of this case has been reduced by integrating 4 additional cameras in a lower angle as described in Section 4.1.4. The second issue is markers that detach

Table 4.1: Recording and post-processing artifacts.

File	Source	Content
`<n>.mcp`	Shogun Live	Full-body recording including the calibrated subject
`<n>.mrec`	MANUS Core	Raw hand recording
`<n>.fbx`	MANUS Core	Exported hand data used for the merge
`<n>_merged.mcp`	Shogun Post	Merged body and hand motion, native working format
`<n>_merged.fbx`	Shogun Post	Merged body and hand motion, exchange format
`<n>/samples.csv`	Ball tracker	Positions, velocities, and timestamps
`<n>/parabolas.csv`	Ball tracker	Accepted trajectory-fit parameters
`<n>/metadata.json`	Ball tracker	Recording configuration

during the take itself. For instance, this can happen while the ball is caught and can also cause the Python script to mistake a detached marker for the ball.

Lastly, for the ball trajectory, the criteria is the plausibility of the fitted model. A take is only counted as usable (for the ball data) if the parameters of the ball are logged.

4.4.4 Limitations

Two limitations of the dataset should be noted. First, the catching motion was performed by a single main performer in order to keep the stylistic variance of the target motion low. The dataset therefore captures the catching strategy of one individual and does not cover inter-personal variation. Second, all takes were recorded at a fixed position within the capture volume, so that locomotion towards the ball is not part of the dataset.

Additionally, since each take was recorded individually, the length of each take may differ.

5 Capturing the Ball Trajectory with Event Cameras

5.1 Introduction to Event Cameras and Reasons for Usage

Traditional frame-based cameras often struggle in dynamic scenarios involving high-speed object motion, such as tracking a thrown ball in real time [8]. Due to fixed frame rates, motion blur, and latency, capturing the precise spatial trajectory of a fast-moving ball presents a significant technical challenge. To overcome these limitations, event-based cameras were introduced into the pipeline. Unlike conventional sensors that capture full images at fixed intervals, event cameras operate asynchronously by measuring per-pixel brightness changes. This architecture offers microsecond-level temporal resolution, a high dynamic range, and minimal latency, making event cameras ideal for tracking dynamic ball trajectories. By continuously capturing log-intensity changes, the system can detect the movement of the ball with very low delay, providing the high-frequency spatial-temporal data necessary for trajectory prediction [20]. Similar event-based approaches have successfully been applied to high-speed object catching in previous work, which served as inspiration for our event-camera-based tracking approach [7].



Figure 5.1: Event camera used for ball trajectory tracking.

5.2 Setup and Tracking Pipeline

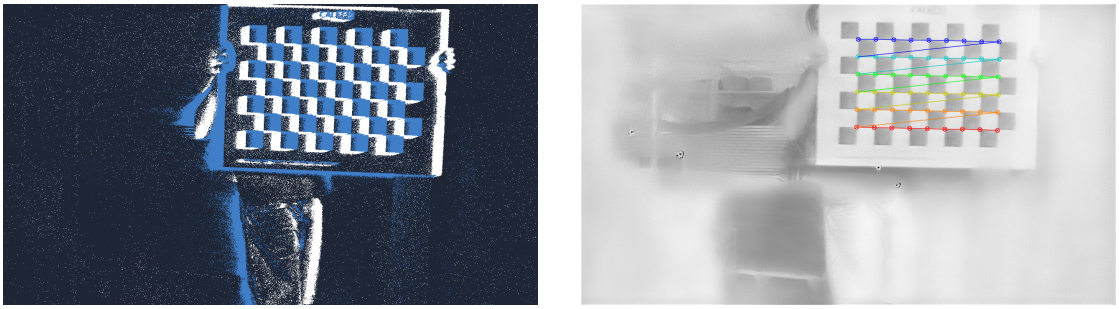
The event camera was positioned next to the robot and observed the ball from the side. In parallel, the existing Vicon motion capture system recorded the position of the ball and was used as ground-truth data for evaluating the position estimates obtained from the event camera.

Several software approaches for accessing and processing the event stream were tested. OpenEB was selected because it provided a simple and reliable way to access the camera data and configure the available sensor filters [17].

To detect the ball in the event stream, DBSCAN was used to group spatially dense events while rejecting isolated events as noise. The center and spatial extent of the detected ball cluster were used to estimate its position and apparent diameter in the image. Together with the ball's known physical diameter and the calibrated camera parameters, these measurements were used to estimate its 3D position relative to the camera. The sequence of estimated 3D positions was then used to approximate the ball's trajectory and predict its future flight path.

5.3 Calibration

The event camera was calibrated before the tracking experiments to determine its camera parameters and account for lens distortion. These parameters were needed to estimate the ball's distance from its apparent size in the image. For the calibration, a checkerboard pattern was recorded with the event camera. The e2calib tool was used to reconstruct grayscale images from the recorded events [19]. The detected checkerboard corners and the known checkerboard pattern were then used to estimate the camera parameters and lens distortion.



(a) Event visualization of the checkerboard.

(b) Reconstructed image with detected checkerboard corners.

Figure 5.2: Checkerboard-based calibration of the event camera.

5.4 Noise Filtering

The event stream also contained events unrelated to the ball, caused by lighting changes, background activity, and sensor noise [15]. In our setup, the infrared light from the Vicon cameras was an additional source of unwanted events. These events formed approximately circular clusters, making them difficult to distinguish from the ball based on shape alone. To reduce this interference and extract the true trajectory of the ball from the raw event stream, spatial and temporal filtering techniques were implemented. The sensor also provides an anti-flicker filter through its Event Signal Processing (ESP) module, which suppresses periodically occurring events generated by flickering light sources [17]. The effect of the anti-flicker filter is illustrated in Figure 5.3.

Additionally, a predefined trigger region was used to initialize tracking when the ball entered the region. The first large circular event cluster appearing inside this region was assumed to be the ball. Restricting initialization to this region reduced the risk of selecting clusters caused by the Vicon cameras.



(a) Unfiltered event stream.



(b) Event stream with anti-flicker filtering.

Figure 5.3: Comparison of the event stream before and after applying the anti-flicker filter.

However, anti-flicker filtering also removed events belonging to the ball, causing its diameter in the image to be estimated inconsistently. This affected the distance estimate, which depended on the ball's known physical diameter and its apparent diameter in the image. An optical filter was therefore added to reduce infrared interference from the Vicon system, reducing the need for aggressive anti-flicker filtering.

5.5 Results

With the help of the IR-filter we successfully designed a pipeline ball tracking and trajectory prediction in 3D space using only one camera. Our accuracy of depth perception of the ball trajectory is now only a few centimeters contrary to the few meters before. This is an improvement on our previous setup without the IR-filter, but this system is still not reliable enough to fully replace our motion tracking setup. There is also the problem caused by our offside camera placement, that our setup is still reliant on positions of the camera and the robot to calculate the relative positions of the ball. We will try to improve the accuracy in the second half of our project. We also plan on positioning the camera on the robot's head. This would resolve our conflict of relative position and dissolve our reliance on the Vicon motion capture system. Our future goal is to have the robot and camera setup be completely independent of the environment.

6 Retargeting Motion Capture Data onto the Unitree G1

Minh Nhat Vu

To process the captured data as input, it needs to be retargeted. Retargeting refers to mapping human movements and joint angles to those of a target robot, presevering the motion as closely as possible. In our project, the target robot is the Unitree G1. For this purpose, we use the General Motion Retargeting project to some extent and adapt its workflow to our requirements. However, retargeting is challenging due to artifacts that can occur during the process. Additionally, the retargeted motions need to be filtered to retain only those of sufficient quality.

6.1 General Motion Retargeting (GMR)

6.1.1 Project Description

Minh Nhat Vu

General Motion Retargeting was originally developed from 2025 onwards by YanjieZe and is publicly available [41]. *GMR* supports real-time retargeting as well as the whole-body teleoperation for 19 different humanoid robots. It also supports several data formats, including `.bvh` or `.fbx`.

The goal of the GMR retargeting process is to convert human motion data into robot motion data. The resulting data consist of the position and rotation for each robot part in each frame. Furthermore, this conversion requires several processing steps.

It starts by defining a mapping between the source human skeleton and robot parts e. g. the torso, head or legs. This is used a setup for the optimization problem for the IK solver. Then, the orientation of the human and robot rest pose gets aligned.

In the next step, we try to get the correct scaling for each robot part based on the height of the source human skeleton. After that, we use these information to solve the optimization problem by minimizing the errors of the body position as well as the orientation. For inverse kinematics, we use *Mink* as a differential IK solver. As a final step, the results are refined by applying additional rotation and translation constraints. These steps are only applicable to a single pose. These methods are applied sequentially to each frame to retarget a motion sequence.

For assessing this retargeting method, a policy was trained on 21 different motions such as walking, dancing and running in IsaacSim using three different setups (**sim**, **simdr**, and **sim2sim**). The success rate was evaluated, with a policy considered successful if the roboter reached the end of the episode without falling.

As a result, most of these motions achieved high success rates above 98 percent, some of them in perfect performance. However, this has considerable tracking errors, both in global and local space. In addition to that, artifacts appeared in some clips such as ground penetration or self-intersection.

Additionally, a user study was performed with 20 participants. They saw various 5 second clips showing the reference motion and the retargeted ones from *GMR*, *Unitree*, *PHC* and *ProtoMotions*. These clips are chosen randomly from 15 of the motions. They were asked which of them are more similar to the reference motion by comparing all 3 methods to *GMR* for every motion. In total, 45 questions were asked. As a result, the participants strongly preferred *GMR* over *PHC* and ProtoMotions. Although the *Unitree G1* was preferred over *GMR*, they found it more difficult to distinguish between the two retargeted motions [3].

6.1.2 From initial approach to final retarget workflow

Minh Nhat Vu

To make use of the GMR project, our initial approach was only to use the component that processes retargeted offline `.fbx` files. The project expects these files to be generated using *OptiTrack*. Because we recorded the source motions in *Vicon Shogun Live* instead, it lead to problems of the axis alignments of the retargeted robot parts, which were corrected later on. Another issue was that the resulting motions were not sufficiently smooth. We attempted to address by applying two methods: a low-pass filter to smooth the motion data and a Kalman filter to reduce noise and compensate for tracking errors. However, achieving satisfactory results remained difficult due to tracking errors that still occurred.

Then we followed a completely different strategy which became our second final approach. For this approach, we added some external steps to the GMR project. First, we extract the skeleton movement from a `.fbx` file using Blender to save the human movements into a `.pkl` file afterwards. Then we use the process of the initial GMR project to retarget for the body without the hands. After that, we map the human joints for the hand onto the Inspire Hands. At the same time, we included the ball movements from the `.csv` files containing the recorded positions and corresponding frame indices. Subsequently, we save the resulting robot movements as well as the ball trajectory in `.npz` format which is used as input for the subsequent reinforcement learning process in *Loco MuJoCo*. Moreover, we also added rendering step that generates an `.mp4` video. It shows the input motion as the FBX skeleton and the retargeted motion simultaneously, with the robot attempting to catch a ball. The robot is additionally displayed from two different viewing angles for visual evaluation. The figure 6.1 shows an example video.

We added an orange-colored area around each leg for the robot because one problem occurred during testing. The hands slightly intersect with the upper part of the legs while the robot is in a rather upright standing position at the beginning of the clip. We assume that this is because the hands come from a different model that is larger than the hands from the original Unitree G1 model. The orange-colored areas are used during the retargeting process to indicate the regions where collisions should be avoided.

As a result, the robot's hands are slightly offset at the begining of the clip, preventing collisions in these regions. After the video is rendered, we assess the data in the final step to determine whether it meets the required quality criteria.

To improve the efficiency of the workflow, we also support retargeting all `.fbx` files in a directory as an input, eliminating the need to run the command separately for each file. During our work, we applied this workflow to multiple sessions containing 50-200 `.fbx` files each, with each recording lasting up to 10 seconds. The complete processing of a session takes only a few minutes.

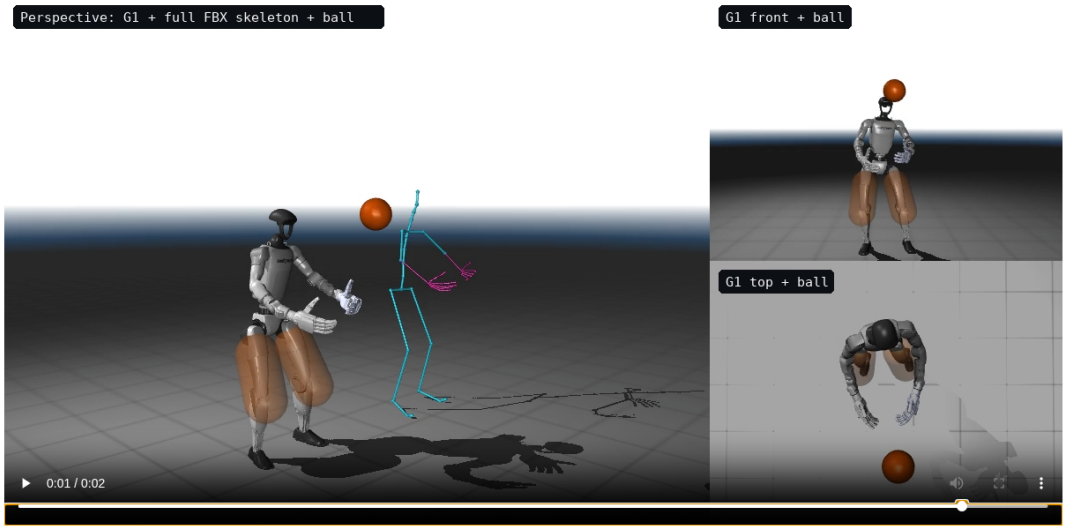


Figure 6.1: MP4 video of human and retargeted movement

6.2 Challenges in Retargeting

Niklas Wasserberg

The challenges in retargeting primarily stem from the difference between the proportions of the human skeleton and the robot's proportions. Furthermore the quality of the retargeted data heavily depends on the quality of the motion capture data the data is based on. Special care needs to be put into ensuring the physical feasibility of the retargeted motions. This section examines each of these problems.

6.2.1 Morphological and Kinematic Differences

The structural discrepancy between human anatomy and the humanoid robot's architecture pose a significant challenge in retargeting. While the human body possesses a high degree of flexibility through it's complex joint structure, humanoid robots are built with rigid bodies. Thus human motions cannot be directly replicated by the robot. In robotics, joints are typically realized by motors with strictly defined, mechanical axis limits and constrained Degrees of Freedom (DoF). Direct mapping from human joints to robot joints is rarely feasible. Discrepancies in limb-length ratios and differing centers of mass mean that identical configurations of the joints yield different poses. Retargeting algorithms must therefore solve an inverse kinematics problem, balancing the target pose with the structural limits of the robot.

6.2.2 Geometrical Artifacts

Even when kinematic constraints are accommodated, direct spatial mapping frequently leads to severe geometrical artifacts. These either arise from noise in the underlying motion capture pipeline or as an byproduct of cartesian scaling:

- **Occlusion:** Optical motion capture systems frequently suffer from marker occlusion during close interactions (e.g. catching an object), resulting in missing data and trajectory discontinuities
- **Self-Collisions:** Due to differences in body volume and segment lengths, motions where human limbs pass close to the torso often cause the robot's rigid geometry to collide with itself.
- **Foot Sliding:** Variation in leg lengths and insufficient contact detection often turns stable human poses into continuous sliding of the robot's feeed across the ground
- **Contact Inconsistencies**: In the context of catching a ball, humans usually explout environmental and self-contact. This can lead to limb interpenetration or to floating contacts.

Mitigating those issues requires geometric post-processing and collision-avoidance constraints.

6.2.3 Physical feasibility

While kinematic retargeting ensures that a motions appears plausible visually, it does not account for underlying dynamics limiting physical execution. A kinematically correct motion could for example damage the robot's hardware:

- **Actuator and Torque Limits:** Actuators operate under strict physical boundaries for maximum torque ($\tau_{\max}$), and velocity ($\dot{q}_{\max}$) . These are often governed by thermal limits and gearbox ratings. Dynamic motions such as sudden swinging can potentially exceed these limits when mapped to a robotic structure.
- **Dynamic Balance:** Human locomotion relies on micro-adjustments to sustain stability. Tracking these movements often leads the robot to a point where it cannot maintain balance.
- **Impact Spikes:** Transitioning from contact-free phases to phases with contact introduce high frequency impact spikes. Without proper trajectory control, these forces can saturate joint drives and damage the robot.

The transition from purely visual imitation to real-world execution requires refinement of the motion using physics-based tracking and trajectory optimization

6.3 Smoothing and Filtering of Retargeting Results

Malte Kaufhold

To avoid the need for extensive reward engineering in the reinforcement learning phase of the pipeline, it may be useful to provide imitation learning with motions that are free of self collisions and jitter and that respect the constraints of joints. One way to realize this is to find better or different methods of retargeting. Another way might be to retarget motion files with current methods and filter out bad results. The validation script implemented is of the latter kind: for each frame of a file it records the number of collisions between joints and which joints were involved. Furthermore, it checks for each frame which joint positions are out of their limits and which joints position jumps between frames in its absolute value more than prescribed by a defined threshold.

6.3.1 General Architecture

The project was implemented in python because of ease of use of the python Mujoco api. The project is organized according to functionality. Each functionality lives in its own python file. Accordingly, there is a file that implements detection of collisions, another for joint jumps between frames and another checking constraints of joints. Then there is one file each for the rendering of the motions, the calling of the different modules and for applying the script to a whole batch of motion files.
The validation script is designed to work with different Mujoco models. It is necessary to expose a .xml model-file.
The dependencies of this project are mujoco and numpy. These can be installed via the requirements.txt file.

6.3.2 Detecting jumps between frames

The Problem of jitter and the (temporary) need to detect it, first time led to the idea of a script to filter out bad motion files and led to a better process from motion capture to retargeting.
Now the need to detect joint jumps is rather irrelevant for current methods of retargeting because of improvement in the retargeting process and the practice of filtering and improving the motion capture files before retargeting begins.

6.3.3 Detecting violations of joint limits

The idea for detecting the violations of joint limits did not stem from a specific problem in our practice, but rather from a paper consulted in the research-process for how to write such a program. [43]
It was still chosen to be implemented, since if a retargeter violates the constraints of a joint, this is inherently wrong, since constraints should not be violated. Because if the model represents the real robot in an accurate manner, the robot itself could not perform these motions.
Whether the problem of violation of joint constraints would have been significant in the imitation learning phase is not known by the author, because it was not tested. But if the retargeted files are to be published it seems cleaner to remove such errors.

6.3.4 Detecting collisions

If a retargeted humanoid robot model collides with itself during simulation it is at least visually unpleasing, since if it resembles the real robot its parts should not penetrate into one another.
The actual significance of the problem in imitation learning remains unknown.

The logic for collision detection takes place in detect_self_collisions.py and helpers.py.
helpers.py provides functions that set the pose of the ball and robot. These functions are also used in another file that renders motions.
Early on, the mistake was made to use two different function in each file. This resulted in a bug being discovered very late. The bug was that quaternion rotations were interpreted differently in the 2 files. This led to wrong collision detection that was not explainable by examining the rendered motion, which used correct interpretations of quaternions. To strengthen the utility of the rendering functionality as a debugging tool (as it was originally intended to be) the functionality for setting the pose of ball and body

in a frame was pulled into separate helper functions set_motion_pose and set_body_pose in helpers.py.
The entry point for the validation script is the function detect_self_contacts. It returns a CollisionSummary that implements a print method and stores important data such as lists of CollisionInfo objects in a dictionary keyed by the frame number. Each CollisionInfo stored this way represents the maximum depth penetration (above a tolerance threshold) of geoms belonging to the same body id in that frame. The CollisionInfo dataclass also stores other information such as the ids of the involved body parts.
Mujoco defines contacts between 2 geoms. But since names are given according to body ids it was decided to group the collisions by body ids. This enables sensible representation of body parts involved in contacts via mujocos mj_id2name() function.[4] For each frame, extract_self_contacts() sets the poses of the robot and ball according to the motion file and runs mj_forward().
After running mj_forward() the contact data is contained in mjData.contact. The program iterates over all contacts and first fetches the corresponding body ids to the geoms in a contact. Then these geoms must belong to different bodyparts, since collision between the same body part cannot be as nicely printed and the depth of the penetration must be above a certain tolerance threshold. Else that contact is ignored.
If there are multiple collisions that were not ignored previously between 2 body parts, which is plausible, since a body part can have multiple geoms associated with it, they are reduced to the collision with the deepest penetration.

6.3.5 npz support

The standard script uses python pickle files for reading data. But a new retargeting project added the need to use the retargeter with npz files. Therefore, a script was implemented that converts these files to pickle files. The idea was at first to just copy the keys and values from the npz files to the pickle files. However, in its entirety this was not possible.
This was due to the fact that the position of the hands is stored with a different key than the positions of the body. But the validation script needs them stored with the same key. It was tried to simply concatenate the degree of freedoms of the body and the hands for each frame. However this led to wrong results, since this was not the position were the model used in the validation script expected these values.
Thus a new file was created in which one can define python dictionaries that store indexes in dof positions for the model used by the validation script by whether these indexes belong to the hand or body.

6.3.6 Inspecting the robot's collisions

Minh Nhat Vu

So far, the validation script does not provide a visual way to detect collisions during the robot's movement. For this reason, we added an inspection tool to display the retargeted motion in a MuJoCo viewer. The tool provides custom keyboard controls for navigating through the motion. These bindings allow to pause, restart, to play in slow motion or go to a specific frame of the clip. It also includes the standard MuJoCo viewer controls for zooming in and out as well as moving or rotating the camera. Figure 6.2 shows this tool with a user interface which consist of the total amount of frames of the clip, the name of the selected file as well as the current timestamp.

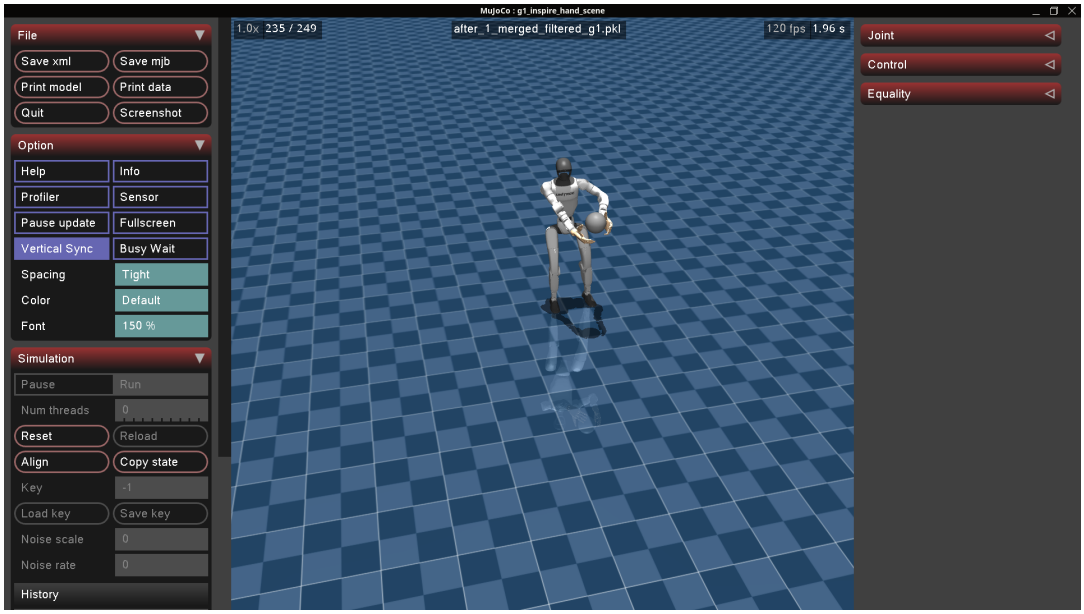


Figure 6.2: Inspection of a robot's movement

6.3.7 Usage during our work

Minh Nhat Vu

In practice, we applied the inspection to identify self-collisions from the retargeted motions of the robot at specific frames. Moreover, we used the validation script to compare the quality in the initial first approach and the final retarget workflow from section 6.1.2. This is used to determine which approach is used later as a result. Because the validation script supports using all `.npz` files in a directory as an input, multiple clips can be evaluated more efficiently.

We used the 60 clips in which the person tried to catch a ball. According to our results, the second one gave us better results than the first one. For the first approach, none of the results are categorized as successful whereas on the second approach one third of the results are successful, and two third of them are not because the hands do not overlap at the beginning.

7 Imitation and Reinforcement Learning with LocoMuJoCo

The previously recorded (see ch. 4) and retargeted (see ch. 6) motion data should now be utilized to train a policy for the Unitree G1. The training itself occurs in a simulated environment as to not damage actual hardware. For this purpose we use the general purpose physics engine MuJoCo (Multi-Joint dynamics with Contact) [26].

7.1 Introduction to LocoMuJoCo

LocoMuJoCo is an imitation learning benchmark tailored for whole-body control and unifies various training and evaluating control policies [1]. Especially advantageous is the support of parallel MJX (MuJoCo XLA) environments accelerating the training process. Furthermore LocoMuJoCo provides built-in domain randomization and through its modular design facilitates the swapping of components such as observations, rewards, etc.. For the imitation learning four JAX algorithms are provided: PPO, GAIL, AMP, and DeepMimic. We utilize the DeepMimic algorithm.

It is also equipped with 22,000+ motion capture datasets, which are retargeted to work with LocoMujoco already, for multiple locomotion tasks including humanoid, quadruped and skeletal human models locomotion. This framework provides for each environment realistic noisy motion data, ground truth expert data and ground truth sub-optimal expert data, which allows LocoMujoco to cover a wide variety of complex movements. Furthermore, this framework allows domain randomization for a better application to real-world robotics and reducing the sim-to-real gap. Regarding the training LocoMujoco supports parameter randomization, such as joint properties, inertia, friction and gravity for more robust training results.

7.2 Defining Environments in LocoMuJoCo

Even though LocoMuJoCo already provides 12 humanoid environments including an environment for the Unitree G1 with 23 degrees of freedom, none fit our robot configuration with 29 degrees of freedom and the additional Inspire Dexterous Hands. For this reason additional environments had to be designed. MuJoCo's native model format is MJCF, an XML-based language designed to describe complex dynamical systems [26]. We slightly modified a preexisting MJCF, that includes the Unitree G1 with 29 degrees of freedom and the Inspire Dexterous hands [5], and added another scene with an additional ball as a free joint. For the DeepMimic algorithm LocoMuJoCo requires sites in the MJCF that serve as the measure of discrepancy between the model's trained and the desired motion. Figure 7.1 shows the Mimic sites of the Unitree G1 with Inspire Dexterous hands.
Based on the existing LocoMuJoCo environment for the Unitree G1 with 23 degrees of freedom we define

a new LocoMuJoCo environment with the modified MJCF, the action space defined in table 7.1 and the observation space defined in table 7.2.

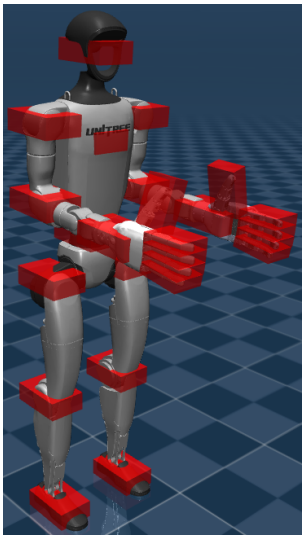


Figure 7.1: Mimic sites of the Unitree G1 with Inspire hands

Index	Joint Name	Min	Max	Index	Joint Name	Min	Max
0	left_hip_pitch_joint	-88	88	20	left_wrist_pitch_joint	-5	5
1	left_hip_roll_joint	-88	88	21	left_wrist_yaw_joint	-5	5
2	left_hip_yaw_joint	-88	88	22	left_thumb_1_joint	-1	1
3	left_knee_joint	- 139	139	23	left_thumb_2_joint	-1	1
4	left_ankle_pitch_joint	-50	50	24	left_index_1_joint	-1	1
5	left_ankle_roll_joint	-50	50	25	left_middle_1_joint	-1	1
6	right_hip_pitch_joint	-88	88	26	left_ring_1_joint	-1	1
7	right_hip_roll_joint	-88	88	27	left_little_1_joint	-1	1
8	right_hip_yaw_joint	-88	88	28	right_shoulder_pitch_joint	-25	25
9	right_knee_joint	-139	139	29	right_shoulder_roll_joint	-25	25
10	right_ankle_pitch_joint	-50	50	30	right_shoulder_yaw_joint	-25	25
11	right_ankle_roll_joint	-50	50	31	right_elbow_joint	-25	25
12	waist_yaw_joint	-88	88	32	right_wrist_roll_joint	-25	25
13	waist_roll_joint	-50	50	33	right_wrist_pitch_joint	-5	5
14	waist_pitch_joint	-50	50	34	right_wrist_yaw_joint	-5	5
15	left_shoulder_pitch_joint	-25	25	35	right_thumb_1_joint	-1	1
16	left_shoulder_roll_joint	-25	25	36	right_thumb_2_joint	-1	1
17	left_shoulder_yaw_joint	-25	25	37	right_index_1_joint	-1	1
18	left_elbow_joint	-25	25	38	right_middle_1_joint	-1	1
19	left_wrist_roll_joint	-25	25	39	right_ring_1_joint	-1	1
				40	right_little_1_joint	-1	1

Table 7.1: Action Space of the Unitree G1 with Inspire hands

Joint Name	Index	Observation Type	Min	Max	Index	Observation Type	Min	Max
root	0-6	FreeJointPos	-inf	inf	66 - 71	FreeJointVel	-inf	inf
left_hip_pitch_joint	7	JointPos	-2.5307	2.8798	72	JointVel	-inf	inf
left_hip_roll_joint	8	JointPos	-0.5236	2.9671	73	JointVel	-inf	inf
left_hip_yaw_joint	9	JointPos	-2.7576	2.7576	74	JointVel	-inf	inf
left_knee_joint	10	JointPos	-0.087267	2.8798	75	JointVel	-inf	inf
left_ankle_pitch_joint	11	JointPos	-0.87267	0.5236	76	JointVel	-inf	inf
left_ankle_roll_joint	12	JointPos	-0.2618	0.2618	77	JointVel	-inf	inf
right_hip_pitch_joint	13	JointPos	-2.5307	2.8798	78	JointVel	-inf	inf
right_hip_roll_joint	14	JointPos	-2.9671	0.5236	79	JointVel	-inf	inf
right_hip_yaw_joint	15	JointPos	-2.7576	2.7576	80	JointVel	-inf	inf
right_knee_joint	16	JointPos	-0.087267	2.8798	81	JointVel	-inf	inf
right_ankle_pitch_joint	17	JointPos	-0.87267	0.5236	82	JointVel	-inf	inf
right_ankle_roll_joint	18	JointPos	-0.2618	0.2618	83	JointVel	-inf	inf
waist_yaw_joint	19	JointPos	-2.618	2.618	84	JointVel	-inf	inf
waist_roll_joint	20	JointPos	-0.52	0.52	85	JointVel	-inf	inf
waist_pitch_joint	21	JointPos	-0.52	0.52	86	JointVel	-inf	inf
left_shoulder_pitch_joint	22	JointPos	-3.0892	2.6704	87	JointVel	-inf	inf
left_shoulder_roll_joint	23	JointPos	-1.5882	2.2515	88	JointVel	-inf	inf
left_shoulder_yaw_joint	24	JointPos	-2.618	2.618	89	JointVel	-inf	inf
left_elbow_joint	25	JointPos	-1.0472	2.0944	90	JointVel	-inf	inf
left_wrist_roll_joint	26	JointPos	-1.97222	1.97222	91	JointVel	-inf	inf
left_wrist_pitch_joint	27	JointPos	-1.61443	1.61443	92	JointVel	-inf	inf
left_wrist_yaw_joint	28	JointPos	-1.61443	1.61443	93	JointVel	-inf	inf
left_thumb_1_joint	29	JointPos	0	1.1641	94	JointVel	-inf	inf
left_thumb_2_joint	30	JointPos	0	0.5864	95	JointVel	-inf	inf
left_thumb_3_joint	31	JointPos	0	0.5	96	JointVel	-inf	inf
left_thumb_4_joint	32	JointPos	0	3.14	97	JointVel	-inf	inf
left_index_1_joint	33	JointPos	0	1.4381	98	JointVel	-inf	inf
left_index_2_joint	34	JointPos	0	3.14	99	JointVel	-inf	inf
left_middle_1_joint	35	JointPos	0	1.4381	100	JointVel	-inf	inf
left_middle_2_joint	36	JointPos	0	3.14	101	JointVel	-inf	inf
left_ring_1_joint	37	JointPos	0	1.4381	102	JointVel	-inf	inf
left_ring_2_joint	38	JointPos	0	3.14	103	JointVel	-inf	inf
left_little_1_joint	39	JointPos	0	1.4381	104	JointVel	-inf	inf
left_little_2_joint	40	JointPos	0	3.14	105	JointVel	-inf	inf
right_shoulder_pitch_joint	41	JointPos	-3.0892	2.6704	106	JointVel	-inf	inf
right_shoulder_roll_joint	42	JointPos	-2.2515	1.5882	107	JointVel	-inf	inf
right_shoulder_yaw_joint	43	JointPos	-2.618	2.618	108	JointVel	-inf	inf
right_elbow_joint	44	JointPos	-1.0472	2.0944	109	JointVel	-inf	inf
right_wrist_roll_joint	45	JointPos	-1.97222	1.97222	110	JointVel	-inf	inf
right_wrist_pitch_joint	46	JointPos	-1.61443	1.61443	111	JointVel	-inf	inf
right_wrist_yaw_joint	47	JointPos	-1.61443	1.61443	112	JointVel	-inf	inf
right_thumb_1_joint	48	JointPos	0	1.1641	113	JointVel	-inf	inf
right_thumb_2_joint	49	JointPos	0	0.5864	114	JointVel	-inf	inf
right_thumb_3_joint	50	JointPos	0	0.5	115	JointVel	-inf	inf
right_thumb_4_joint	51	JointPos	0	3.14	116	JointVel	-inf	inf
right_index_1_joint	52	JointPos	0	1.4381	117	JointVel	-inf	inf

right_index_2_joint	53	JointPos	0	3.14	118	JointVel	-inf	inf
right_middle_1_joint	54	JointPos	0	1.4381	119	JointVel	-inf	inf
right_middle_2_joint	55	JointPos	0	3.14	120	JointVel	-inf	inf
right_ring_1_joint	56	JointPos	0	1.4381	121	JointVel	-inf	inf
right_ring_2_joint	57	JointPos	0	3.14	122	JointVel	-inf	inf
right_little_1_joint	58	JointPos	0	1.4381	123	JointVel	-inf	inf
right_little_2_joint	59	JointPos	0	3.14	124	JointVel	-inf	inf
ball	60-66	BodyPos	-inf	inf	125 - 130	BodyVel	-inf	inf

Table 7.2: Observation Space of the Unitree G1 with Inspire hands

7.3 Learning on self-recorded data

LocoMuJoCo supports the datasets AMASS, LAFAN1, and their own defalut dataset. Additionally an implementation for custom datasets is provided, that takes preprocessed trajectories as inputs. We modified the custom dataset implementation to receive raw .npz-files and convert them to trajectories automatically to facilitate the usage of our data for the imitation learning.

7.4 Incorporating Ball Trajectories into the Imitation Learning Process

First we tried to model the ball as an additional free joint in the environment, since this enabled realistic physical simulation of the contact dynamics of the ball. However the interaction of the additional free joint lead to instability during the LocoMuJoCo training process. Without the ball in the simulation, the imitation learning was already quite successful, yielding mean episode length around 100 seconds as well as a mean cumulative reward of around 100. Adding the ball as an inter-actable free joint to the simulation would heavily decrease the mean episode length to 5 seconds and the mean cumulative reward to 4.
Since the free joint causes instability, we opt to model the ball as a simple body that does not interact with the environment, but purely serves as an observation. To realize this, we overwrite the static body position for the ball-body with the recorded ball trajectory. By making small changes to the LocoMuJoCo framework we also ensured that the ball trajectory was used during training.
This change makes it possible, to achieve stable imitation learning while considering ball positions and velocities. This causes the trained policy to actually target the ball by making the robot reach for the observed ball position. However, actual interaction with the ball is currently not considered. This issue needs to be resolved when refining the policy through reinforcement learning.

7.5 Imitation Learning Results

Training the Unitree G1 with the Dexterous Inspire Hands on a Dataset of 150 motion files with the environment parameters displayed in table 7.3 yields a policy that imitates the motion data well. Depending on the balls trajectory, the robot reaches for the ball differently as demonstrated by the motion data.
Issues that occur in the trained policy are heavy shaking of the robots arms and legs. Also since the

retargeted motion is imperfect, the policy copies these imperfections such as suddenly bending its wrist and not fitting the ball accurately with its hands. Another issue that has to be mentioned is, that the robot did not interact with the ball during the learning process but only copies the demonstrations and assumes that the ball will not act differently. Therefore, imitation learning is not sufficient for deployment.
To resolve the shaking of the robot arms and legs we employed domain randomization displayed in 7.4. Even though the robot still shakes, the shaking is a lot less significant than prior to the domain randomization.

7.6 From Imitation Learning to Reinforcement Learning

LocoMuJoCo not only provides a framework for imitation learning but also for reinforcement learning with the PPO algorithm using JAX. Since pure reinforcement learning yields poor, not human-like results, we want to provide a warm start, by loading the policy trained through imitation learning. However, when using large learning rates, it is quite likely, that the previously learned policy is overwritten in early stages of reinforcement learning. To prevent this, we utilize Kullback-Leibler regularization, which penalizes a new policy when being to far from an old policy.
Important factors for utilizing further reinforcement learning, are enabling interaction between the robot and the ball during training and diversifying the throws seen during training. This requires exchanging

Table 7.3: DeepMimic Environment Hyperparameter Configuration

Hyperparameter	**Value**
Environment Name	MjxUnitreeG1BallInspire
Horizon	1000
Terminal State Type	RootPoseTrajTerminalStateHandler
Control Type	Torque Control
Reward Type	MimicReward
Hidden Layers	[512, 256]
Learningrate	0.0001
Number of Environments	2048
Number of Steps	200
Total Timesteps	$300 \cdot 10^6$
Update Epochs	4
Number of Minibatches	32
γ	0.99
GAE λ	0.95
Clipping ϵ	0.2
Initial std	0.2
Learnable std	false
Activation	tanh
Weight Decay	0.0001
Number of Seeds	1

Table 7.4: DeepMimic Domain Randomization Parameters

Parameter	Value
Randomized Gravity Range	[9.66, 9.96]
Randomized tangential Geometry Friction Range	[0.92, 1.08]
Randomized Joint Damping Range	[0.6, 1.4]
Randomized Joint Armature Range	[0.08, 0.12]
Randomized additional Base Mass Range	[-1.0, 1.0]
Randomized multiplied Link Mass Range Root Body	[0.95, 1.05]
Randomized multiplied Link Mass Range other Bodies	[0.95, 1.05]
Added Joint Position Noise Scale	0.0015
Added Joint Velocity Noise Scale	0.03

the MJCF model, that only provides the ball as a geometry, with the MJCF model, that includes the ball as a free joint. Since the environment with action and observation space has to remain the same to utilize the pretrained imitation learning policy, only the path to the MJCF file is updated through the training configuration.

7.6.1 Kullback-Leibler Regularization

Kullback-Leibler (KL) Regularization penalizes a new policy according to the KL divergence, which measures how far the new policy diverges from the old policy [38]. The KL divergence between two policies π_1 and π_2 that associate different distributions over actions a to each state s is defined as $KL(\pi_1||\pi_2)(s) = \int_{-\infty}^{\infty} \pi_1(s,a) \log(\frac{\pi_1(s,a)}{\pi_2(s,a)})da$.
This divergence is calculated between the original policy resulting from the imitation learning step and the current policy of the reinforcement learning step. To punish strong divergence from the original imitation learning policy, we take the mean of the KL divergence from all states and scale it with a predefined coefficient. The resulting value is added to the total loss of the current reinforcement learning step.

7.6.2 Introducing diverse Ball Throws

The goal of introducing diverse ball throws when resetting the environment is to improve generalization capabilities of the policy. Therefore we define a custom initial state handler, that sets the robot to a random start orientation at the worlds origin around its z-axis with randomly sampled angle θ_r. Additionally, a start pose and velocity of the ball are computed. To get a ball throw that is attainable for the robot to catch, we randomly sample the initial distance d_b between the ball and the robot in a range of 2.5 to 7.5 meters. The initial ball height z_s lies between 1.1 and 1.7 meters. Furthermore, we introduce variability in the balls incoming direction by offsetting the ball by an angle θ_b between $-\pi/4$ and $\pi/4$ from the robots heading.

From these random samples we calculate the global pose of the robot and the ball:

$$\mathbf{p}_r = [x_r, y_r, z_r] = [0.0, 0.0, 0.79] \tag{7.1a}$$

$$\mathbf{q}_r = [\cos\frac{\theta_r}{2}, 0.0, 0.0, \sin\frac{\theta_r}{2}] \tag{7.1b}$$

$$\mathbf{p}_b = [x_b, y_b, z_b] = [x_r + d_b \cdot \cos(\theta_r + \theta_b), y_r + d_b \cdot \sin(\theta_r + \theta_b), z_b] \tag{7.1c}$$

To create an acceptable ball trajectory, the initial ball velocity needs to be set, such that the ball will pass through a point $\mathbf{p}_t$ in front of the robots chest. The target height z_t is randomly sampled in an interval between 0.8 and 1.05 meters. From this sampling the target position of the ball is calculated as:

$$\mathbf{p}_t = [x_t, y_t, z_t] = [x_r + 0.35 \cdot \cos\theta_r, y_r + 0.35 \cdot \sin\theta_r, z_t] \tag{7.2a}$$

From this target pose, the initial velocity $\mathbf{v}_b$ can be calculated by randomly sampling the time t_b that the ball takes from its start position $\mathbf{p}_b$ to the target position $\mathbf{p}_t$ between 0.8 and 1.4 seconds.

$$\mathbf{v}_b = [v_x, v_y, v_z] = [\frac{x_t - x_b}{t_b}, \frac{y_t - y_b}{t_b}, \frac{z_t - z_b}{t_b} + 0.5 \cdot 9.81 \cdot t_b] \tag{7.3a}$$

Finally random noise is added to the initial ball velocity $\mathbf{v}_b$, to introduce additional variability to the balls trajectory.

7.6.3 Fitting Rewards to the Catching Process

For the task of catching the reward function has to evaluate, how well the robots hands are fitted around the ball and if the ball is actually being held. The robot itself should only move its root as much as necessary and avoid high torque-actions.

Evaluating the Hands Fit to the Ball

To achieve a human-like catching behavior, the robot should fully grasp the ball with its fingers and not just squeeze the ball within its palms. The goal of this reward is to fit the fingers and palms as close to the exterior of the ball as possible. Therefore, a list of all distances between all finger and palm geometries to the ball geometry are calculated from the simulation. These distances are stacked into separate vectors $\mathbf{d}_{fingers}$ and $\mathbf{d}_{palms}$. From these vectors we calculate reward vectors $\mathbf{r}_{fingers}$ and $\mathbf{r}_{palms}$ as:

$$r_{fingers,i} = \exp(-w_{fingers} \cdot d^2_{fingers,i}) \tag{7.4a}$$

$$r_{palms,i} = \exp(-w_{palms} \cdot d^2_{palms,i}) \tag{7.4b}$$

Where the entries of the reward vectors decrease as the absolute value of distances increase. The speed of decrease is scaled by the weights $w_{fingers}$ and w_{palms}.
The final rewards are computed by taking the maximum entry of the reward vector respectively:

$$r_{fingers} = \max_i(r_{fingers,i}) \tag{7.5a}$$

$$r_{palms} = \max_i(r_{palms,i}) \tag{7.5b}$$

Measuring if the Ball is being held

For the task of catching a ball it s especially important to check, whether the ball is actually being held by the robot. Therefore, we check how many of the finger and palm geometries are in contact with the ball by summing over the collision booleans of these geometries with the ball geometry s_{col}. The contact reward is scaled by the contact weight $w_{contact}$:

$$r_{contact} = s_{contact} \cdot w_{contact} \cdot s_{col} \tag{7.6a}$$

To evaluate wether the ball is actually stopped and held by the robots hands, the velocity of the ball is considered. To quantify this measurement, we take the negative exponential of the mean squared linear ball velocity scaled with $w_{ballvel}$.

$$r_{ballvel} = \exp\left(-w_{ballvel} \cdot \frac{1}{3}(v_{ball,x}^2 + v_{ball,y}^2 + v_{ball,z}^2)\right) \tag{7.7a}$$

Finally an additional hold reward is computed by combining the palm reward, the finger reward, and the ball velocity reward:

$$r_{hold} = w_{hold} \cdot r_{palms} \cdot r_{fingers} \cdot r_{ballvel} \tag{7.8a}$$

Punishing large Robot Motion

High root velocities and high torques are undesirable for the task of catching a ball. For this reason, we punish large linear and angular root velocities. Therefore, we calculate the velocity reward as follows:

$$r_{vel} = \exp\left(-w_{vel} \cdot \frac{1}{3}(v_{root,x}^2 + v_{root,y}^2 + \omega_{ball,z}^2)\right) \tag{7.9a}$$

This equation yields low returns for high linear velocities of the root in x or y direction and angular velocities about the z-axis.
To avoid high torques we add an action cost that subtracts the mean squared action value from the final reward scaled by w_{action}.

$$c_{action} = w_{action} \cdot \frac{1}{n} \sum_i a_i^2 \tag{7.10a}$$

Combined Reward

The final reward sum is combined as follows:

$$\begin{aligned} r = \; & s_{palms} \cdot r_{palms} \\ & + s_{fingers} \cdot r_{fingers} \\ & + s_{contact} \cdot r_{palms} \\ & + s_{ballvel} \cdot r_{ballvel} \\ & + s_{hold} \cdot r_{hold} \\ & + s_{vel} \cdot r_{vel} \\ & - s_{action} \cdot c_{action} \end{aligned} \tag{7.11}$$

The sum weights $s_{palms}, s_{fingers}, s_{contact}, s_{ballvel}, s_{hold}, s_{vel}$, and s_{action} are used to set the importance of each reward term.

7.6.4 Reinforcement Learning Results

Even after several modifications to LocoMuJoCo the resulting policy from further reinforcement learning is unsuccessful. The agent seemingly unlearns the previous policy regardless of the KL regularization and the robot does not survive very long before falling to the ground. For this reason we decide to abandon training with LocoMuJoCo and instead pursue training with another Repository called MjLab (see chapther 9), where results are much more promising.

8 Reinforcement Learning with Gymnasium via MuJoCo

In this chapter we tried to train a policy with trial and error of simulated throws and explicitly did not use imitation learning or captured motion tracking data in our training. We used Gymnasium and MuJoCo for simulating and training our policy.

8.1 Introduction to Gymnasium

For humanoid robots with large observation spaces and action spaces, traditional reinforcement learning are often not sufficient. Therefore, Deep Reinforcement Learning (DRL) with Deep Q-Networks (DQN) is preferred for humanoid learning. Gymnasium [27] provides an open source standard API for handling DRL environments, which are utilized by many Reinforcement Learning libraries such as Stable-Baselines3 [18]. It also enables vectorized hardware acceleration, enabling faster learning.
There are eleven already implemented MuJoCo [26] environments provided by Gymnasium, as well as a superclass for MuJoCo based environments. For the realistic simulation of the Unitree G1 a physics engine such as MuJoCo is necessary to compute the behavior of the robot accurately.

8.2 Creating Gymnasium-Environment

Creating a MuJoCo based Gymnasium-Environment can be done by simply providing the MJCF file and the desired observation space to the initialization of the superclass for MuJoCo based environments. Additionally, the step function and the reset function need to be implemented. For the step function, the simulation based on the chosen action is already implemented by the superclass. We have to provide the new reward and observation, as well as whether the episode was terminated or truncated, based on the new state of the environment.
For the Unitree G1 we use the observation space represented in table 7.2. These observations can be extracted from the MuJoCo data from qpos for the joint positions and qvel for the joint velocities. The design of the reward function depends on the desired task and will be explained in detail in the following sections.
An episode is terminated, when the robots root position falls below a threshold along the worlds z-axis. Truncation is always set to false.

8.3 Designing Reward-Functions

Designing effective reward functions for high-dimensional continuous control requires balancing primary task objectives against dynamic stability constraints. Unbounded or ill-conditioned reward terms

frequently induce undesirable physical behaviors, such as high-frequency joint jitter or extreme actuator saturation.

8.3.1 Two-Layer Reward Architecture

To decouple low-level dynamic stability from high-level task objectives, the reward function is structured into a two-layer hierarchy:

$$\mathscr{R}_{\text{total}} = \mathscr{R}_{\text{lower}} + \mathscr{R}_{\text{upper}}$$

8.3.2 Lower-Level Regularization ($\mathscr{R}_{\text{lower}}$)

Focuses on robot health, joint smoothness, and energy efficiency:

- Upright Torso Orientation (r_{healthy}): Evaluates the z-component of the pelvis world-frame orientation vector ($\mathbf{R}_{3,3} \in [-1, 1]$) to keep the trunk vertical.
- Foot Contact Consistency (r_{feet}): Evaluates active contact pairs between the floor geometry and the foot links, penalizing single-foot lifting during standing tasks.
- Control Regularization ($r_{\text{small_ctrl}}$): Computes the mean-squared normalized control action $r_{\text{small_ctrl}} = -\frac{1}{N}\sum a_i^2 \in [-1, 0]$ to enforce energy-efficient torque commands.
- Smoothness & Jerk Reduction (r_{smooth}): Penalizes joint accelerations normalized by physics timestep, clipped to prevent numerical explosion:

$$r_{\text{smooth}} = -\text{clip}\left(\frac{\frac{1}{N}\sum \ddot{q}_i^2}{(1/\Delta t)^2}, 0.0, 1.0\right)$$

- Grounded Position & Posture Drift ($r_{\text{joint_dev}}$, $r_{\text{position_drift}}$): Implements Gaussian kernels $r = \exp(-\sigma \cdot \|\mathbf{x}\|^2)$ that smoothly penalize deviation from target joint postures and (x, y) world coordinates without generating massive negative gradient cliffs during sudden evasive maneuvers.

8.3.3 Upper-Level Task & Curriculum Learning ($\mathscr{R}_{\text{upper}}$)

When extending the environment to dynamic tasks like catching an incoming object (G1CatchWithHandsEnv), multi-phase curriculum scheduling gradually activates task-specific objectives:
Phase 0: Base standing policy preservation ($\mathbf{w}_{\text{task}} = 0$).
Phase 1 (Perception Phase): Ball observation inputs fade in without active arm movement rewards.
Phase 2 (Pre-positioning & Approach): Activates spatial hand-to-ball distance guidance ($r_{\text{hand_to_ball}}$) while arm damping remains high to suppress violent flailing.
Phase 3 (Full Catch Execution): Activates velocity matching ($r_{\text{hand_vel_match}}$), palm surface orientation alignment ($r_{\text{hand_orient}}$), and terminal catch bonuses ($\mathscr{R}_{\text{catch}} = +100$).

8.4 Challenges and Results

8.4.1 Starting with PPO

Before training a model to catch the ball, we firstly needed to train a policy to just stand in the initial position. To do this we trained using the PPO algorithm. However, these early trials quickly revealed fundamental hurdles, as the robot survived for only about 0.6 seconds or 300 simulation steps after 16 hours of training before collapsing to the ground. This observation highlighted the necessity to temporarily shift focus away from the complex catching motion toward basic standing and balancing. In the following weeks, technical issues such as local system crashes hampered progress during longer training sessions. Through targeted revisions of the reward functions, the robot's survival time incrementally increased to 1.5 seconds.

8.4.2 Switching to SAC

A systematic comparison of different learning algorithms revealed that Proximal Policy Optimization (PPO) struggled to achieve pure standing stability, with the robot consistently tipping over after a few seconds. Switching to Soft Actor-Critic (SAC) yielded significantly more stable results, enabling the robot to remain upright for the complete 20 seconds of simulation. Integrating an optimized robot model further reduced unnatural jitter, removed sliding away from the center and drastically lowered the fall rate.

8.4.3 Final Results

In the final phase of the project, the focus shifted toward transferring this pre-trained standing policy into an expanded environment featuring ball interactions. Although loading the baseline policy into the ball environment succeeded after initial setbacks, extended training sessions exhibited catastrophic forgetting, where the robot suddenly lost its ability to balance after several hours of training. From these experimental series, we concluded that pure Reinforcement Learning without structural guidance—such as reference motion data from Imitation Learning—reaches its limits, as the high-dimensional action space required for ball catching is too complex to reliably converge through trial and error alone.

9 Reinforcement Learning and Motion Imitation using mjlab for Unitree Robots

9.1 Introduction to mjlab

Mjlab [40] is a framework for GPU-accelerated reinforcement learning for robots that combines MuJoCo's physics simulation [26], using the MuJoCo Warp GPU backend, with an Isaac Lab inspired composable API. Observations, actions, rewards, termination conditions, and curriculums are defined as reusable components [40]. The policy is trained using Proximal Policy Optimization (PPO) [21] through RSL-RL [22] and Figure 9.1 illustrates the workflow.

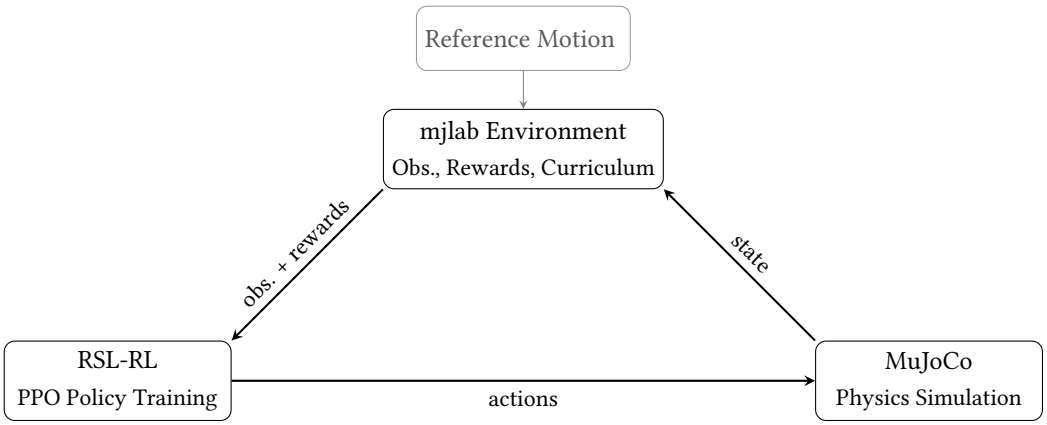


Figure 9.1: Relationship between mjlab, MuJoCo and RSL-RL. The reference motion defines the desired behavior, mjlab formulates the RL task, MuJoCo simulates the robot, and RSL-RL optimizes the control policy using e.g. PPO.

In this work, mjlab is used to implement a two-stage training pipeline for ball-catching tasks on the Unitree G1 with Inspire Hands. The first stage learns to reproduce reference motion of human demonstrations. The second stage further tunes the resulting policy of stage one using fully synthetic procedurally generated ball throws. Unitree provides a deployment adapter for policies trained using mjlab [31].

9.2 Defining the G1 Environment

The environment contains the Unitree G1 with Inspire Hands and a dynamic spherical ball entity. The ball has a diameter of 15 cm, a mass of 112 g and physically interacts with both the robot and the terrain.

The policy outputs 41 joint-position actions to control the robot body as well as the hand joints. Simulated contact sensors additionally provide information about interactions between the ball and the hands for reward calculation.

Table 9.1: Actor observations and their dimensions.

Actor observation	**Dimensions**
Base angular velocity	3
Projected gravity	3
Joint positions	41
Joint velocities	41
Previous action	41
Ball position in the torso frame	3
Ball velocity relative to the torso, expressed in its frame	3
Ball visibility	1
Total	**136**

Both stages share the same actor architecture and observations. The actor receives its previous action, its own state, and ball observations if available. The critic can see additional information, specifically the reference pose.

The terrain starts out flat and then gradually introduces random millimeter-scale bumps to make the robot more robust against real-world, non-perfect floors. During motion imitation, the ball approximately follows the recorded trajectory by initializing its velocity using the first few tracked positions.

9.3 Motion Imitation

The re-targeting output, as described in Chapter 6, provides reference body position, rotation, and velocity for motion tracking rewards. Mjlab contains an implementation of the motion-tracking component of *BeyondMimic*, which formulates imitation as a reinforcement learning task [40, 9]. The policy is rewarded for matching the reference poses and velocities of the tracked bodies. In our adaptation, the privileged critic receives the reference motion as observations, while the actor does not. We resample our re-targeting output to 50 Hz (from 120 Hz) and align our data with mjlab's conventions (e.g., quaternion order and joint names).

As explained in the previous section, our ball does not follow the reference motion's path exactly. Because of that, we added additional rewards already in the Motion Imitation phase that encourage catching the simulated ball. That is a small reward for decreasing distance between the hands and the ball and a larger reward for actually touching the ball with the palms.

9.4 Reinforcement Learning for Catching

The second stage transfers the actor network from the imitation stage. The critic and optimizer are recreated because the critic's observations and the task change. The policy is then trained using synthetic ball throws, with rewards and penalties outlined in Section 9.4.1.

Synthetic throws vary their origin and target position. A curriculum expands the throw distance from 2 m to 5 m and the direction from $\pm 15^{\circ}$ to $\pm 75^{\circ}$. The required hold time is also increased. This should improve the policy's abilities over time to generalize over a wider range of catches than what was recorded by the human demonstrator.

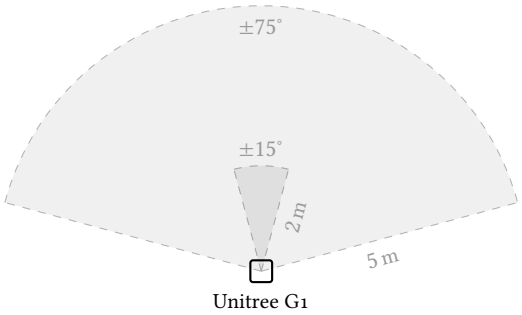


Figure 9.2: Top-Down View. Origins and target positions are randomized, while distance and angular range is increased.

Successful termination requires the ball to contact both palms with low velocity for an increasing duration. If the robot falls, the ball touches the floor, or on timeout (e.g., when the robot holds the ball up but not using the hands), we also terminate.

The policy is implemented using a simple MLP with $512-512-128$ hidden units and ELU activations.

9.4.1 Designing Reward Functions

Besides common stability and robustness rewards, we use three rewards to guide optimization. A small reward providing feedback for initial ball proximity, a larger reward for actually touching the ball only with the desired elements, and an even larger retention term to increase hold duration.

Table 9.2: Reward groups and their intended behavior.

Reward group	**Intended behavior**
Standing posture	Maintain balance
Interception	Bring palms toward the ball
Palm and finger contact	Encourage palm contact and finger closing
Stable grasp	Keep the ball in the intended pose for some time
Terminal success	Reward completion
Torque, joint limit, collision, motion penalties	Discourage costly or undesirable behavior

The following equations describe three selected reward terms before applying their individual weights.

Interception The interception reward encourages the hands to approach the ball while matching its velocity:

$$r_{\text{intercept}} = \exp\left(-\frac{d^2}{\sigma_d^2} - \frac{v_{\text{rel}}^2}{\sigma_v^2}\right). \tag{9.1}$$

Where, d is the distance between the ball center and the midpoint between the palms. The relative speed v_{rel} is the difference between the ball velocity and the average hand velocity. The parameters σ_d, σ_v control how quickly the reward decreases with distance and relative speed. This term is active only after the ball is launched. Matching velocities encourages a controlled interception rather than statically hitting the ball.

Palm contact For each hand, we define a contact indicator $c_L, c_R \in \{0, 1\}$. It is one if the ball makes an accepted contact with the corresponding palm surface, and zero otherwise.

$$r_{\text{contact}} = \frac{c_L + c_R}{2}, \tag{9.2}$$

Stable grasp The stable-grasp reward requires the ball to remain close to the hands, move slowly relative to them, and maintain palm contact. For a launched ball, it is defined as

$$r_{\text{grasp}} = \begin{cases} \exp(-E_{\text{pose}}), & d < d_{\max}, \quad v_{\text{rel}} < v_{\max}, \quad \text{palm contact maintained,} \\ 0, & \text{otherwise.} \end{cases} \tag{9.3}$$

Here, d is the distance from the ball center to the midpoint of the palms, and v_{rel} is the ball's speed relative to the average hand velocity. The thresholds $d_{\max}$ and $v_{\max}$ define when the ball is considered stable and slow enough to be held.

The pose error determines how well the grasp matches the desired catching posture:

$$E_{\text{pose}} = \frac{d_{\text{zone}}^2}{\sigma_{\text{zone}}^2} + \frac{\|\Delta \mathbf{p}_{xy}\|^2}{\sigma_{xy}^2} + \frac{(\Delta p_z - h_{\text{target}})^2}{\sigma_z^2} + \frac{\sin^2 \theta}{\sin^2 \theta_{\max}}. \tag{9.4}$$

The distance d_{zone} is zero inside the rectangular catch region and measures the distance to that region outside it. The vector $\Delta \mathbf{p}$ points from the palm midpoint to the ball center, expressed in world coordinates. The horizontal component $\Delta \mathbf{p}_{xy}$ aligns the palm midpoint with the ball, while the vertical term encourages the palms to support it from below. The angle θ measures the torso's tilt from upright. The parameters σ and $\theta_{\max}$ control the tolerance of these pose errors.

9.5 Challenges and Results

Initially, the contact reward treated the entire hand as a valid contact surface. The policy then learned to catch the ball with the sides of its hands rather than the palms. After restricting valid palm contacts using custom sensors in the environment, this was quickly fixed.

We then defined a rectangular catch region box in front of the robot's torso and only counted successful catches if the ball was inside of it, to encourage the robot to keep its hands in front of its body instead of attempting catches to the sides or above its head, which is something we noticed later on without it. This box is also useful for a second limitation: Our Vicon-based ball tracking fails to track the ball once it is occluded by the robot's hands, and thus we would either need to approximate the current position or mark it as invisible in the observation. To represent this during tracking, we hide the ball once it reaches this region. The ball remains physically simulated but is not visible to the actor network. In our preliminary

tests, this only slightly negatively affected success rates and could likely be improved by using a GRU network or by supplying an approximated ball trajectory parabola instead of the real-time position.

During the second training stage, the policy network quickly deviates from the demonstrated movements while successfully improving catching performance. Because we want the resulting catches to still look ”human”, we need to address this trade-off. This could, for example, be achieved by mixing in the old rewards from phase 1 from time to time.

After 6 hours of training on an RTX 4080, the observed success rate was nearly 100% for the simulated throws.

10 Unitree RL Lab on top of Isaac Lab

10.1 Introduction to Isaac Lab

An alternative to training in MuJoCo is to use NVIDIA Isaac Sim simulation as training environment with the tools of NVIDIA Isaac Lab. In this chapter we describe our implementation approaches of the ball catching task in Isaac Lab.

10.1.1 Isaac Sim and Isaac Lab

To understand the implementations we first give a brief introduction to the Isaac Sim and Isaac Lab framework. Isaac Sim is a robotics simulator by NVIDIA within the NVIDIA Omniverse platform. It uses the PhysX physics engine and runs simulations on the GPU. Isaac Lab is an open-source robot learning framework that uses Isaac Sim as simulation backend. It is the successor of Isaac Gym [14] and provides the needed environments, sensors, actuator models, domain randomization and interfaces to RL libraries. For the used repository unitree_rl_lab, Isaac Sim 5.1.0 and Isaac Lab 2.3.0 are required.

Isaac Lab provides several types of workflows for implementing RL tasks. The direct workflow implements the whole environment in one class, while the manager-based workflow builds the environment using exchangeable building blocks in configuration classes.

10.1.2 Manager-based Environments

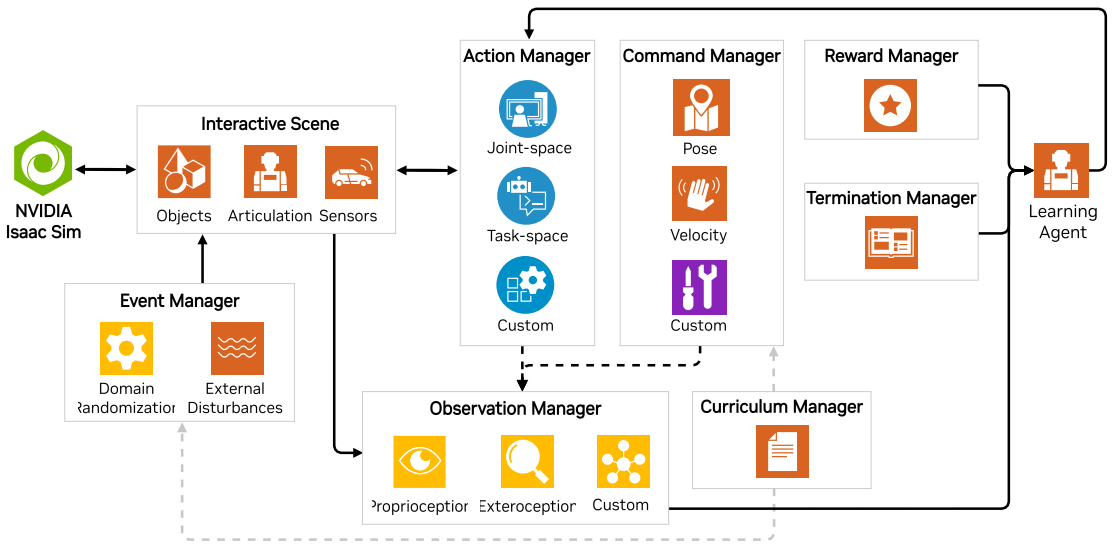


Figure 10.1: Manager-based task workflow in Isaac Lab. Source: Isaac Lab documentation [25].

The manager-based workflow is recommended by NVIDIA and is used in the unitree_rl_lab repository. The main idea is to define the environment in a configuration class instead of imperative code. Each aspect of the RL problem is taken over by its own manager, which is configured with terms. A term is a function plus parameters, so for example a reward term is a reward function with a weight. A benefit of this approach is the reusability of terms between tasks. New tasks can be created by combining existing terms and adding new functions. The managerial approach also provides possible separation between policy and critic observations for asymmetric actor-critic algorithms.

The interaction of the managers with the simulation and the agent is shown in figure 10.1. An overview of the managers and their functions for the ball catching task is given in table 10.1.

Manager	Role	Example in our task
Scene	provides assets and sensors	G1, the ball, contact sensor
Action	maps policy output to actuator commands	joint position targets
Observation	computes observation groups (actor-critic), adds noise, keeps history	ball and hand positions, joint states
Reward	management of reward terms	eg. distance hand to ball, catch bonus, posture penalties
Termination	ends episodes with failure, success, timeout	triggers when robot falls, ball caught, ball dropped
Event	triggers events, randomization at startup, reset, etc.	friction, masses, random pushes
Command	task commands that change over time	reference motion, ball throw
Curriculum	adapts the task during training	easier throws at the beginning

Table 10.1: Managers of a manager-based RL environment in Isaac Lab and their use in our ball catching task.

10.2 The unitree_rl_lab Framework

unitree_rl_lab is an open-source repository by Unitree Robotics themselves, which provides Isaac Lab environments for their robots and deployment code [30]. The fork we are using is based on the state of November 2025 where Isaac Sim 5.1.0 and Isaac Lab 2.3.0 are used. Figure 10.2 shows the components of the repository and their use in our project.

The repository already provides a pipeline from training to deployment on the robot we use with many needed assets. We can use the given G1 configuration of the mimic task, which defines for every joint a PD controller with gains derived from the rotor inertia of its motor, together with the motor's torque and velocity limits. The mimic task implements motion tracking for example motions but we can use it as a template with our own motion data. There are scripts for training with RSL-RL PPO, replaying a checkpoint, exporting the policy as ONNX and for converting motion data. For deployment, a C++ program runs the exported policy and when doing sim-to-sim, it controls the MuJoCo simulator or when doing sim-to-real, it is connected via DDS to the real G1 robot (see ch. 11). Yet, the repository has limitations for our task and requires adaptations in most of the components.

Still, we decided to build on unitree_rl_lab instead of starting from scratch with Isaac Lab, because of the already implemented mimic task, the given deployment chain, the original G1 assets and that it has been successfully tested on the real robot.

Path	Content	Status
unitree_rl_lab/		
source/…/unitree_rl_lab/		
assets/robots/	robot configurations	used
tasks/locomotion/	velocity tasks	not used
tasks/mimic/	motion tracking (BeyondMimic)	template
tasks/ball_catching/	Phase 1, Phase 2 and Unified task	new
utils/	export of the deploy configuration	used
scripts/rsl_rl/	training and replay, ONNX export	extended
scripts/mimic/csv_to_npz.py	conversion of CSV motions to .npz	replaced
scripts/mimic/pkl_to_npz.py	conversion of GMR .pkl to .npz	new
deploy/	C++ deploy program	used
unitree_rl_lab.sh	launcher for training and replay	used

Figure 10.2: Structure of unitree_rl_lab and the parts we use, compared to the original state of November 2025.

10.3 Integration of the Ball Catching Task

We implemented the ball catching task as a task package in a unitree_rl_lab fork. We extended the tasks with own terms for the project-specific logic. The task package contains three tasks: Phase 1, a motion tracking task for imitation learning, Phase 2, the reinforcement learning task for ball catching, and the unified version that combines both tasks in one. For now we ignore the dexterous hands and only use the wrists as catchers. We model the ball as a rigid sphere in PhysX and throw it with a custom command term. We set the physics simulation to 200 Hz, the policy to 50 Hz and use 4096 parallel environments for training with an asymmetric actor-critic PPO algorithm.

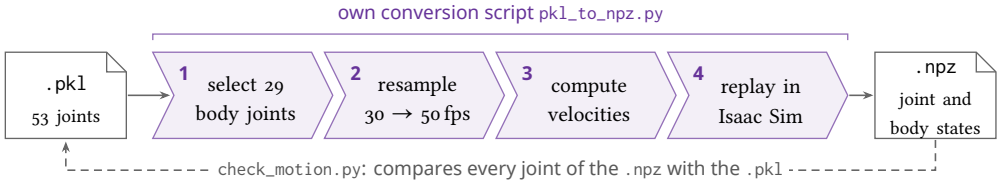
All three task use the same observation layout. We pad the slots of a task that cannot fill with zeros, so that we can transfer checkpoints between the tasks, see figure 10.4. During the development process we found that to be a drawback, because every change in the observation layout has to be mirrored in both tasks. We encountered several bugs because of this, so in conclusion this choice is a trade-off between reusability and maintainability. We express the ball and hand observations as relative to the robot in its yaw frame, so they do not depend on the position in the simulation and can furthermore be computed from motion capture data on the real robot. We added an observation that predicts the position and time of the ball at catch height. We only give the policy the data that is also available to a real robot so we add noise. The critic gets privileged simulation data, e.g. the ball velocity and the robot base state.

10.3.1 Motion Data Preparation

To train the Phase 1 motion tracking task, we first need to convert the retargeted motion data to the expected format. The motion data comes as retargeted GMR motions in .pkl files with 53 joints including the hands (see ch. 6). The mimic task of unitree_rl_lab expects .npz files with joint and body states at 50 Hz. So we replaced the existing .csv-to-.npz conversion with our own script, see figure 10.3. There we select the 29 body joints, resample to 50 fps, compute the velocities and replay the motion in Isaac Sim to obtain the poses of all bodies. A check script compares every joint of the .npz with the source file. We added this check after we found that the first conversion took the wrong joint columns from the left hand, which is shown in figure 10.3 a).



(a) Joint columns of the GMR output (G1 with hands, 53 joints)



(b) Conversion steps

Figure 10.3: Preparation of the motion data for the mimic task.

10.4 Training Approaches

Up to this point we implemented the two training approaches, sequential and unified task. The Phase 1 task does imitation learning as reinforcement learning. We give rewards for tracking the reference motion and use the existing BeyondMimic implementation [9] from the mimic task of unitree_rl_lab (Basics to BeyondMimic (see ch. 9)). The episode starts there in the reference state at a sampled time of the clip. We sample sections in which the robot often fails more often, so that the policy can learn to recover.

In Phase 2, we train catching without a motion reference. We place the robot standing with a random heading at the start and the ball 1.0–2.5 m in front of it. We launch the ball after a random delay on a ballistic path towards the robot's chest. The flight time is estimated to be 0.3–0.6 s. The episode ends when the ball is either caught, dropped, missed, the robot loses balance or a time-out occurs. We reward the approach of the ball, the catch and holding posture, but reduce the posture rewards during training. Our curriculum makes the throws harder over time.

The combination of the two phases can be done either sequentially or in our unified task. For the sequential approach, we start Phase 2 from the actor of Phase 1. The problem there is, that the zero-filled inputs differ between the phases and that the learned motion can be forgotten. Therefore, we prefer the

unified task and keep the sequential variant as comparison. In the unified task, we combine the tracking and ball rewards in one task, as in DeepMimic [16].

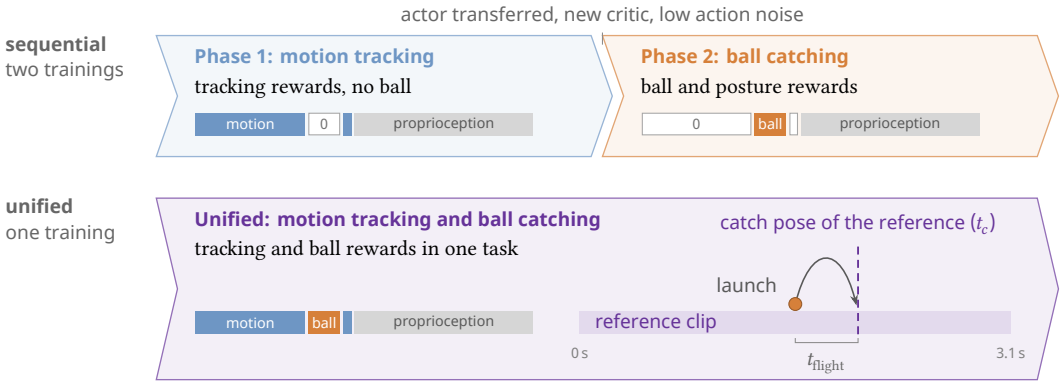


Figure 10.4: Two ways to combine motion tracking and ball catching.

- Inputs to scale; 0: slots filled with zeros, which change between the two phases
- Unified: the ball is launched t_{flight} = 0.3–0.6 s before the catch pose of the reference (t_c, placeholder 2.0 s)

10.5 Course of the Work and Challenges

During the course of the work we encountered several challenges. In May, we familiarized ourselves with the unitree_rl_lab repository, the general manager-based structure of Isaac Lab and we looked into the LATENT paper [42] for improvement inspiration. While setting up Isaac Sim and Isaac Lab on our machines, we had to fix interface changes between versions of Isaac Lab and RSL-RL. Furthermore, a graphics driver bug prevented the simulation from running, which resulted in us using the simulation headless temporarily.

In June and July, we implemented our own tasks Phase 1 and Phase 2 and the transfer between them. There we ran into several errors that prompted us to analyze if using a merged task is better than the sequential approach. Errors that do not crash the training revealed themselves to be hard to find, especially in headless mode, even though we added several tracking metrics and visualizations while headless training. We encountered such errors in the coordinate frames, the geometry of the throw, in the scaling between the phases, in the reward and termination logic, and in the motion data. For example, our ball command threw the ball again before the drop and miss terminations could react, because Isaac Lab standardly updates the commands after checking the terminations [24]. As a result, we learned the lesson to implement automatic checks for every pipeline step and to do thorough visual checks of the results early on.

In August, we analyzed and corrected many of the errors in the first Phase 2 version and created the unified task. In September, we did the first training of Phase 1 on a recorded clip and continued with code review and correction of the motion conversion.

10.6 Outlook

The following things are planned for the next steps of the project. We will continue to train the unified task as well as the sequential approach and compare the results. For that we will train the Phase 2 task first time with a warm start from Phase 1 but also from scratch. As a metric we will compare the catch rate.

We will alter the task designt to improve the learning and the catch rate. For that we will aim the ball at the hand position instead of the chest. Further reward changes will be tested for optimization purposes. We try to adopt suitable extensions from LATENT, e.g. a reward for smooth actions and noise on the root state. We still need to adapt the size and mass of the ball to the real ball model we use in real life. After achieving a good catch rate in simulation, we will add the dexterous hands to our task.

To improve the sim-to-real transfer, we will add more domain randomization. Noise, delay and dropouts on the ball observations will help to simulate the filtered Vicon data. Delayed actions will be added as well as a more realistic throw distribution fitted to real throws recorded by Vicon.

We need to implement the ball and hand observation into the existing C++ deploy program of unitree_rl_lab, which will be fed with live Vicon data. We will also focus on the sim-to-sim test in MuJoCo before the real robot deployment.

For reward tuning we will look into automatic hyperparameter optimization. AutoML methods like early stopping or population-based training might help finding better reward weights, curriculum and hyperparameters.

11 Deployment of Policies

Deployment is the process of transferring the trained policy of the G1 robot onto the robot itself. The robot can then run the trained policy on its own, without the training environment. Two ways of deploying policies have been used to *test* (Sim-to-Sim) and to *verify* (Sim-to-Real) the trained policy.

11.1 Sim-to-Sim

A trained policy from one simulation environment can be tested in another simulation environment. We use this to test a trained policy before deploying it onto the physical robot. Every simulator has its own physics engine and numerical details. A policy that only works because of one simulator's quirks will usually fail when moved to another simulator. If the behavior stays the same in both simulators, it is much more likely that the physical robot will behave the same way too. Running the policy on another simulator therefore removes the factors that are specific to one simulation environment. Two simulation environments were used during the project: **MuJoCo** (see ch. 7) and **Isaac Lab** (see ch. 10).

11.1.1 Isaac Lab to MuJoCo

Transferring a policy from Isaac Lab to a robot in MuJoCo was done with the official Unitree RL Lab repository [30], which includes a deploy program that runs the robot policy in MuJoCo. This approach worked, but also needed some configuration. First and foremost, a real Linux machine is required. This is because the Unitree fork of MuJoCo only runs on Linux and because WSL and virtual machines in general must to pass USB input through, which proved to be difficult to solve on WSL.

The deploy program needs two files for each policy. The first is the trained network parameters, which is exported once the trained policy is replayed in Isaac Lab. It is exported as `policy.onnx` into the `exported` directory, wherever the trained parameters are located. The second file is a configuration file, `deploy.yaml`. It describes the environment as the deploy script needs it: the observations and their scales, the action scaling, the default joint positions, the PD gains and joint order. The `deploy.yaml` file is found in `config` inside of the deploy program directory. Policies are also located here. The main configuration file found under `config/config.yaml` then decides which policy directory is loaded.

The deploy program itself is written in C++ and has Unitree SDK2 [33] as runtime dependency. Policies are uploaded over network interfaces, in case of Sim-to-Sim, we just use a the loopback `lo` interface as upload target. The policy is not copied onto the robot. Instead, the deploy program runs the policy on the host computer. It reads the robot state and sends resulting motor commands over a network interface. For MuJoCo, the simulator `unitree_mujoco` [32] takes the place of the robot. The simulator needs a working gamepad and it we also enabled virtual elastic band. The band holds the robot upright while it is initialized.

The deploy program is controlled with the gamepad through a small state machine. The robot starts in a passive, damped state. With `L2 + Up` it moves into a fixed standing pose and is then lowered to the

ground in the simulator. With `R1 + X` the policy takes control, and the elastic bank can be released. `L2 + B` returns the robot to the passive state at any time.

11.1.2 MuJoCo to Isaac Lab

TODO

11.2 Sim-to-Real

Once the policy behaves as expected in MuJoCo, it is deployed on the physical G1 robot. This step *verifies* the policy by revealing whether the remaining differences between simulation and reality were covered well enough during training.

11.2.1 Isaac Lab to Real

The setup and the policy files are the same as described in Section 11.1.1. The same deploy program, the same `policy.onnx` and the same `deploy.yaml` are used without changes. The steps only diverge when the deploy program is started. Instead of the loopback interface, the network interface that is connected to the robot is chosen. Before that, the robot's on-board motion control has to be disabled, so only the deploy program sends motor commands. The deploy program refuses to start is another program is already controlling the motors.

The policy is started the same as in simulation. Since there is no elastic band on the real robot, the robot is secured during the standing phase and the first seconds of the policy.

11.2.2 MuJoCo to Real

TODO

12 Conclusion

Bibliography

[1] Firas Al-Hafez, Guoping Zhao, Jan Peters, and Davide Tateo. 2023. LocoMuJoCo: A Comprehensive Imitation Learning Benchmark for Locomotion. 6th Robot Learning Workshop, NeurIPS.

[2] Ivan Alberico, Marco Cannici, Giovanni Cioffi, and Davide Scaramuzza. 2025. Egocentric Event-Based Vision for Ping Pong Ball Trajectory Prediction. arXiv:2506.07860 [cs.CV] https://arxiv.org/abs/2506.07860

[3] Joao Pedro Araujo, Yanjie Ze, Pei Xu, Jiajun Wu, and C. Karen Liu. 2025. Retargeting Matters: General Motion Retargeting for Humanoid Motion Tracking. arXiv:2510.02252 [cs.RO] https://arxiv.org/abs/2510.02252

[4] DeepMind Technologies Limited. n.d.. Computation. https://mujoco.readthedocs.io/en/stable/computation/index.html Accessed: 2026-09-25.

[5] Fred Feraco. 2026. Unitree G1 + Inspire Hand Assets (MuJoCo). https://github.com/feraco/unitree_g1_inspire Last accessed: 27.08.2026.

[6] Benedek Forrai, Takahiro Miki, Daniel Gehrig, Marco Hutter, and Davide Scaramuzza. 2023. Event-based Agile Object Catching with a Quadrupedal Robot. arXiv:2303.17479 [cs.RO] https://arxiv.org/abs/2303.17479

[7] Benedek Forrai, Takahiro Miki, Daniel Gehrig, Marco Hutter, and Davide Scaramuzza. 2023. Event-based Agile Object Catching with a Quadrupedal Robot. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, Piscataway, NJ, USA, 12177–12183. doi:10.1109/ICRA48891.2023.10161392

[8] Guillermo Gallego, Tobi Delbruck, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J Davison, Jörg Conradt, Kostas Daniilidis, and Davide Scaramuzza. 2022. Event-based vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 1 (2022), 154–180. doi:10.1109/TPAMI.2020.3008413 Accessed: 2026-09-25.

[9] Qiayuan Liao, Takara E. Truong, Xiaoyu Huang, Yuman Gao, Guy Tevet, Koushil Sreenath, and C. Karen Liu. 2025. BeyondMimic: From Motion Tracking to Versatile Humanoid Control via Guided Diffusion. arXiv:2508.08241 [cs.RO] https://arxiv.org/abs/2508.08241

[10] MANUS Meta. n.d.. MANUS Core: Getting Started. https://docs.manus-meta.com/3.2.0/Software/Getting%20Started/ Documentation version 3.2.0. Last accessed: 18.09.2026.

[11] MANUS Meta. n.d.. MANUS Core: Recording. https://docs.manus-meta.com/3.2.0/Software/Core/UI/recording/ Documentation version 3.2.0. Last accessed: 18.09.2026.

[12] MANUS Meta. n.d.. MANUS Core: Remote Trigger (UDP). https://docs.manus-meta.com/2.5.0/Software/Rec/Remote%20trigger/ Documentation version 2.5.0. Last accessed: 18.09.2026.

[13] MANUS Meta. n.d.. Shogun Post Integration: Using. https://docs.manus-meta.com/3.2.0/Plugins/Shogun/using/ Documentation version 3.2.0. Last accessed: 18.09.2026.

[14] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, et al. 2025. Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. arXiv:2511.04831 [cs.RO] https://arxiv.org/abs/2511.04831

[15] Vandana Padala, Arindam Basu, and Garrick Orchard. 2018. A noise filtering algorithm for event-based asynchronous change detection image sensors on TrueNorth and its implementation on TrueNorth. *Frontiers in Neuroscience* 12, Article 118 (2018), 14 pages. doi:10.3389/fnins.2018.00118 Accessed: 2026-09-25.

[16] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Transactions on Graphics* 37, 4, Article 143 (2018), 14 pages. doi:10.1145/3197517.3201311

[17] Prophesee. n.d.. Event Signal Processing. https://docs.prophesee.ai/stable/hw/manuals/esp.html#chapter-esp. Accessed: 2026-09-21.

[18] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8. https://jmlr.org/papers/v22/20-1364.html

[19] Robotics and Perception Group. n.d.. E2Calib: How to Calibrate Your Event Camera. https://github.com/uzh-rpg/e2calib Accessed: 2026-09-27.

[20] Kanon Sato, Takuya Nakabayashi, Masahiro Yamaguchi, Kyota Higa, Ryo Fujiwara, and Hideo Saito. 2024. Time-consistent Ball Tracking and Spin Estimation with Event Camera. In *Proceedings of the 7th ACM International Workshop on Multimedia Content Analysis in Sports*. Association for Computing Machinery, New York, NY, USA, 59–64. doi:10.1145/3689061.3689067 Accessed: 2026-09-25.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 doi:10.48550/arXiv.1707.06347

[22] Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller, and Marco Hutter. 2025. RSL-RL: A Learning Library for Robotics Research. arXiv:2509.10771

[23] Arth Shukla, Kehlani Fay, Huaze Liu, Darin Anthony Djapri, Alexiy Buynitsky, and Nicklas Hansen. 2026. Catch It All!: Generalizable Dynamic Catching with a Dexterous Hand. 4th RSS Workshop on Dexterous Manipulation: Scalable Learning for Human-Level Skills. https://openreview.net/forum?id=O2gDb6qzSx

[24] The Isaac Lab Project Developers. 2025. Isaac Lab Source Code, Release v2.3.0: `ManagerBasedRLEnv` and `RewardManager`. https://github.com/isaac-sim/IsaacLab/blob/v2.3.0/source/isaaclab/isaaclab/envs/manager_based_rl_env.py Files `envs/manager_based_rl_env.py` and `managers/reward_manager.py`. Accessed: 2026-10-05.

[25] The Isaac Lab Project Developers. 2025. Reference Architecture – Isaac Lab Documentation (v2.3.2). https://isaac-sim.github.io/IsaacLab/v2.3.2/source/refs/reference_architecture/index.html Accessed: 2026-10-05.

[26] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, IEEE, Piscataway, NJ, USA, 5026–5033. doi:10.1109/IROS.2012.6386109

[27] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. 2024. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032

[28] Unitree Robotics. n.d.. Unitree Documentation Center. https://support.unitree.com/home/en/developer/ Accessed: 2026-09-28.

[29] Unitree Robotics. n.d.. Unitree G1. https://www.unitree.com/g1/ Accessed: 2026-09-28.

[30] Unitree Robotics. n.d.. Unitree RL Lab. https://github.com/unitreerobotics/unitree_rl_lab Accessed: 2026-09-24.

[31] Unitree Robotics. n.d.. Unitree RL Mjlab. https://github.com/unitreerobotics/unitree_rl_mjlab

[32] Unitree Robotics. n.d.. unitree_mujoco. https://github.com/unitreerobotics/unitree_mujoco Accessed: 2026-09-24.

[33] Unitree Robotics. n.d.. unitree_sdk2. https://github.com/unitreerobotics/unitree_sdk2 Accessed: 2026-09-24.

[34] Vicon. 2021. HSL Scripting with Vicon Shogun. https://help.vicon.com/download/attachments/13341545/HSL%20scripting%20with%20Vicon%20Shogun.pdf 5 July 2021. Last accessed: 18.09.2026.

[35] Vicon. 2023. Vicon Vantage Reference Guide. https://help.vicon.com/download/attachments/15052377/Vicon%20Vantage%20Reference%20Guide.pdf Last accessed: 18.09.2026.

[36] Vicon. 2025. Getting Started with Vicon Shogun Live. https://help.vicon.com/download/attachments/376704362/Getting%20started%20with%20Vicon%20Shogun%20Live.pdf Shogun Live 1.17, revision 1, 4 September 2025. Last accessed: 18.09.2026.

[37] Vicon. 2026. Vicon DataStream SDK 1.13.0 Developer's Guide. https://help.vicon.com/download/attachments/726007858/Vicon%20DataStream%20SDK%20Developer%27s%20Guide.pdf February 2026. Last accessed: 18.09.2026.

[38] Nino Vieillard, Tadashi Kozuno, Bruno Scherrer, Olivier Pietquin, Rémi Munos, and Matthieu Geist. 2021. Leverage the Average: an Analysis of KL Regularization in RL. arXiv:2003.14089 [cs.LG] https://arxiv.org/abs/2003.14089

[39] Momoe Yamane, Masahiro Yamaguchi, Kyota Higa, Ryo Fujiwara, and Hideo Saito. 2025. Real-Time Ball Tracking and Action Classification using an Event Camera. In *Proceedings of the 8th International ACM Workshop on Multimedia Content Analysis in Sports (MMSports '25)*. ACM, New York, NY, USA. doi:10.1145/3728423.3759404

[40] Kevin Zakka, Qiayuan Liao, Brent Yi, Louis Le Lay, Koushil Sreenath, and Pieter Abbeel. 2026. mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning. arXiv:2601.22074 https://arxiv.org/abs/2601.22074

[41] Yanjie Ze, João Pedro Araújo, Jiajun Wu, and C. Karen Liu. 2025. GMR: General Motion Retargeting. https://github.com/YanjieZe/GMR Last accessed: 25.09.2026.

[42] Zhikai Zhang, Haofei Lu, Yunrui Lian, Ziqing Chen, Yun Liu, Chenghuai Lin, Han Xue, Zicheng Zeng, Zekun Qi, Shaolin Zheng, Qing Luan, Jingbo Wang, Junliang Xing, He Wang, and Li Yi. 2026. Learning Athletic Humanoid Tennis Skills from Imperfect Human Motion Data. arXiv:2603.12686 [cs.RO] https://arxiv.org/abs/2603.12686

[43] Qingrui Zhao, Kaiyue Yang, Xiyu Wang, Shiqi Zhao, Yi Lu, Xinfang Zhang, Wei Yin, Qiu Shen, Xiaoxiao Long, and Xun Cao. 2026. Make Tracking Easy: Neural Motion Retargeting for Humanoid Whole-body Control. arXiv:2603.22201 [cs.RO] doi:10.48550/arXiv.2603.22201